esprit
Se former autrement

# 2023 - 2024

# GRADUATION PROJECT

## NATIONAL ENGINEERING DEGREE

## SPECIALTY : INFORMATION TECHNOLOGY

# Design & Development of an Admin Panel for an E-commerce Platform


**By:** *Ahmed Abid*

**Academic supervisor:** Ms Nadia Chagtmi

**Corporate Internship Supervisor:** Mr Sami Kammoun


converty
.shop

cdio

EUR-ACE

Délivrée par: Cti

CONFÉRENCE DES GRANDES ÉCOLES
EXCELLENCE FOR A COMPLEX WORLD

J'autorise l'étudiant à faire le dépôt de son rapport de stage en vue d'une soutenance.

Encadrant professionnel, **Mr Kammoun Sami**

**Signature**




J'autorise l'étudiant à faire le dépôt de son rapport de stage en vue d'une soutenance.

Encadrant académique, **Mrs. Nadia Chagtmi**

**Signature**

# Dedication

This work is dedicated to:

**My family**, whose unwavering support and countless sacrifices have been crucial to my success. I hope this work reflects the fruits of your sacrifices and makes you proud, as that is immensely important to me.

**My dear friends**, who have consistently believed in me, offering their support through difficult times. I hope for a future for you that is filled with joy and accomplishments. Your belief in me is invaluable.

**To all those who have supported me**, my gratitude is immense for your encouragement; and to those who doubted me, thank you for pushing me to unleash my full potential against all odds. This is incredibly meaningful to me.

**And finally, to all those who contributed to making this project possible**, I dedicate this work to you.

**Ahmed Abid**

# Acknowledgments

I would like to express my sincere gratitude to all the individuals and organizations who played a crucial role in the success of my internship.

I want to extend special thanks to Mr. Sami Kammoun, my supervisor at Converty, for his warm welcome and invaluable mentorship throughout my internship. His dedication and expertise have been instrumental in my professional growth and achievements.

I am also deeply grateful to Ms. Nadia Chagtmi, my academic supervisor at ESPRIT, for her insightful guidance, support, and patience during my internship. Her advice and oversight have greatly influenced my approach and understanding of our field.

I would also like to thank Converty, the host organization, for providing a supportive and dynamic environment that has contributed significantly to my learning experience and professional development.

# Preface

This internship report documents my experience at Converty, an e-commerce platform, during my third year of engineering studies at ESPRIT (École Supérieure Privée d'Ingénierie et de Technologies). The internship lasted for six (6) months and provided me with insights into the field of software engineering and its impact on the company's clients.

Converty is a dynamic e-commerce startup located in Jardins Menzah 2, Tunisia, that is revolutionizing the online business landscape in Tunisia. It offers a unique combination of CRM and CMS solutions, empowering businesses to effectively manage and expand their online presence.

As a CMS, Converty simplifies online store creation, customization, product management, and integration with analytics platforms like Meta, Google, and Microsoft. As a CRM, Converty has strategic partnerships with top Tunisian delivery companies and fulfillment centers, an extensive order management system, and real-time analytics for performance monitoring.

This report provides a detailed analysis of my internship experience at Converty, highlighting the challenges faced, the solutions implemented, and the valuable lessons learned. It emphasizes the importance of software engineering principles in optimizing company processes and fostering innovation in the ever-changing world of e-commerce.

# Abstract

Modern software design and development practices emphasize the need for fast, reliable, and secure applications. Consequently, the adoption of caching mechanisms, decoupling of services, and modular design is becoming increasingly popular.

At Converty, we are committed to adopting best practices and design patterns to ensure optimal performance and user satisfaction. We have designed and developed an admin panel that allows users to manage orders, products, and categories efficiently and intuitively. This platform addresses several critical problems faced by small and medium-sized enterprises (SMEs), including difficulty in managing e-commerce operations, lack of synchronization with delivery companies, and challenges in managing user permissions.

To achieve these objectives, we implemented a series of functional requirements, such as real-time updates and user management with micro-permissions. Additionally, we ensured that the platform meets stringent non-functional requirements, including performance, scalability, reliability, availability, security, and a user-friendly interface.

The development process incorporated Scrum to adapt to the dynamic needs of the project, ensuring that our small team could effectively manage and deliver the project goals. By tailoring Scrum practices to our specific context, we enhanced collaboration and productivity.

**Keywords:** Software Design, Development, Scrum, Optimization, Admin Panel, E-commerce, Agile, Real-time Updates, User Management.

# Résumé

Les pratiques modernes de conception et de développement de logiciels mettent l'accent sur le besoin d'applications rapides, fiables et sécurisées. Par conséquent, l'adoption de mécanismes de mise en cache, de découplage des services et de conception modulaire devient de plus en plus populaire.

Chez Converty, nous nous engageons à adopter les meilleures pratiques et les modèles de conception pour garantir des performances optimales et la satisfaction des utilisateurs. Nous avons conçu et développé un panneau d'administration qui permet aux utilisateurs de gérer les commandes, les produits et les catégories de manière efficace et intuitive. Cette plateforme répond à plusieurs problèmes critiques rencontrés par les petites et moyennes entreprises (PME), notamment la difficulté de gestion des opérations de commerce électronique, le manque de synchronisation avec les entreprises de livraison et les défis liés à la gestion des permissions des utilisateurs.

Pour atteindre ces objectifs, nous avons mis en œuvre une série d'exigences fonctionnelles, telles que les mises à jour en temps réel et la gestion des utilisateurs avec des micro-permissions. De plus, nous avons veillé à ce que la plateforme réponde à des exigences non fonctionnelles strictes, y compris la performance, la scalabilité, la fiabilité, la disponibilité, la sécurité et une interface utilisateur conviviale.

Le processus de développement a intégré Scrum pour s'adapter aux besoins

dynamiques du projet, garantissant que notre petite équipe puisse gérer et atteindre efficacement les objectifs du projet. En adaptant les pratiques Scrum à notre contexte spécifique, nous avons amélioré la collaboration et la productivité.

**Mots-clés :** Conception de logiciels, Développement, Scrum, Optimisation, Panneau d'administration, Commerce électronique, Agile, Mises à jour en temps réel, Gestion des utilisateurs.

# Contents

# List of Figures

# List of Tables

# List of Code Snippets

# List of Acronyms

- **API** = **A**pplication **P**rogramming **I**nterface
- **AWS** = **A**mazon **W**eb **S**ervices
- **CDN** = **C**ontent **D**elivery **N**etwork
- **CMS** = **C**ontent **M**anagement **S**ystem
- **CRM** = **C**ustomer **R**elationship **M**anagement
- **CRUD** = **C**reate, **R**ead, **U**pdate, **D**elete
- **GCP** = **G**oogle **C**loud **P**latform
- **GKE** = **G**oogle **K**ubernetes **E**ngine
- **JWT** = **J**son **W**eb **T**oken
- **SDK** = **S**oftware **D**evelopment **K**it
- **UI** = **U**ser **I**nterface

# General Introduction

The MENA region has witnessed remarkable growth in the e-commerce industry over the last ten years. However, the development and legalization of online payment services have not kept pace with this growth, resulting in a significant gap between demand and availability. To address this gap, cash-on-delivery (COD) has emerged as the dominant payment method, driving the expansion of the sector.

E-commerce platforms like Shopify have become synonymous with online retail, particularly in the dropshipping industry. However, this focus on one dominant player has created a demand for specialized applications that can enhance Shopify's core capabilities to better meet the specific needs of different markets. Despite its flexibility, Shopify can become cumbersome and slow due to its extensive features. Moreover, it has a steep learning curve, making it less accessible for beginners. Additionally, the platform hosts numerous apps, each addressing a specific use case and requiring its own subscription fee, which can be a significant barrier for small businesses. Furthermore, the high level of abstraction in Shopify makes it difficult to customize the platform extensively to suit individual business requirements.

In response to these challenges, Converty was conceived with a unique value proposition aimed at maximizing user efficiency and customization. Converty empowers users to create fast, highly converting, and customizable online stores. It enables complete business management from a single platform—eliminating the need for coding, spreadsheets, extraneous apps, and the associated headaches. distinctive offering that focuses on optimizing user productivity and customization. Converty empowers users to effortlessly create fast, highly converting, and customizable online stores. It provides a comprehensive platform for managing all aspects of a business, eliminating the need for coding, spreadsheets, additional applications, and the associated challenges.

For that reason, Converty has to be created in a state of the art manner, with the best practices in software development and with the rapid expansion of the platform in mind. This report details the internship experience at Converty, focusing on the implementation of a comprehensive CRM & CMS system. The report is structured as follows:

The first chapter of this report details the scope of the internship, detailing the COD state of the art, the problems to be solved and the objectives of the internship with actionable items.

The second chapter provides an overview of the project setup, getting into the intricacies of engineering decisions made during the internship. It also gets into edge details of the project environment such as the infrastructure and the DevOps practices in place.

Each following chapter represents a user journey that should be met and is structured with the requirements, design, implementation, validation, and a summary of the accomplishments and the issues faced during the implementation of the user journey.

We conclude the report with a reflection on the internship experience, the lessons learned, and the recommendations for future work.

*Chapter 1*

# Scope Statement

## 1.1 Host Company

Converty is a Tunisian startup founded in January 2024 that provides a comprehensive e-commerce platform for small to medium-sized enterprises (SMEs).



**Figure 1.1:** Converty Logo

The platform offers a range of features, including online store management, customer relationship management (CRM), marketing automation, and reporting tools. Converty aims to simplify the e-commerce experience for SMEs by providing an integrated solution that streamlines operations, reduces costs, and improves efficiency.

## 1.2 State of the Art - COD Scene

Popular e-commerce platforms like Shopify, WooCommerce, and Magento dominate the market, offering robust solutions for online retail. However, these platforms often present challenges such as limited customization options, high costs, and complex user interfaces. These issues can be particularly burdensome for small to medium-sized enterprises (SMEs) that lack the resources to fully leverage these tools.

### 1.2.1 Software as a Service (SaaS) Platforms

Software as a Service (SaaS) platforms like Shopify and BigCommerce have gained popularity in recent years due to their ease of use and low barrier to entry. These platforms offer a range of features which can be useful for SMEs, such as built-in payment processing, inventory management, and marketing tools. However, SaaS platforms often come with high

monthly fees and limited customization options, making them less appealing for businesses looking to scale.

### 1.2.2 Open Source Platforms and Content Management Systems (CMS)

Open source platforms like WooCommerce and Magento offer more flexibility and customization options than SaaS platforms, making them popular choices for larger enterprises. However, these platforms can be complex to set up and maintain, requiring technical expertise that many SMEs lack. Additionally, open source platforms often require additional plugins and integrations to achieve the desired functionality, which can add to the overall cost and complexity of the system.

## 1.3 Problem Statement

The inspiration for this project stemmed from observing these challenges firsthand and conducting extensive market research. Conversations with small business owners revealed a clear need for a more integrated, user-friendly platform that could streamline their e-commerce operations. This feedback shaped the initial vision and identified the following problems:

- Difficulty in managing orders, products, and categories efficiently
- Lack of synchronization between orders and delivery companies, leading to delays and errors
- Challenges in managing users and teams with appropriate permissions
- Inability to see updates in real-time, causing delays in decision-making and operations

## 1.4 Project Requirements

### 1.4.1 Functional Requirements

To achieve this goal, we have identified the following functional requirements:

- Manage orders, products, and categories
- Synchronize orders with delivery companies
- Manage users and teams with micro-permissions
- See updates in real-time (Real-time connection)

### 1.4.2 Non-Functional Requirements

In addition to the functional requirements, the platform must also meet the following non-functional requirements:

- Performance
- Scalability
- Reliability
- Availability
- Security
- Friendly user interface

## 1.5 Project Deliverables

- **Source Code**

  At the end of this internship, the source code of the platform will be delivered. This includes all the code written for the front-end, back-end, and any other components of the platform. The code will be well-documented and organized to facilitate future development and maintenance as well as tested and reviewed to ensure quality and reliability.

- **Documentation**

  A comprehensive set of documentation will be provided to better understand the

architecture and design of the platform. This includes technical documentation, installation instructions, diagrams and flowcharts of the system.

- **Requirements Specification**

  A detailed requirements specification document will be delivered, outlining the functional and non-functional requirements of the platform. This document will serve as a reference for future development and enhancements.

- **Final Report**

  A final report will be submitted at the end of the internship, summarizing the project objectives, methodology, results, and recommendations. This report will provide insights into the development process and key learnings from the project.

- **Presentation**

  During this internship, a progress report in the form of a presentation was delivered on a monthly basis. At the end of the internship, a final presentation will be given to showcase the platform, its features, and the development process.

## 1.6 Project Management Plan

Before we dive into the development process, it is essential to establish a project management plan to guide the team throughout the internship. Let's start by defining the context of the methodology that was used and related to the "Software Development Life Cycle" (SDLC).

### 1.6.1 Software Development Life Cycle (SDLC)

The Software Development Life Cycle (SDLC) is a process used by software development teams to design, develop, test, and deploy software applications. The SDLC consists of several phases, each with its own set of activities and deliverables. The phases of the SDLC are shown in Figure ??.

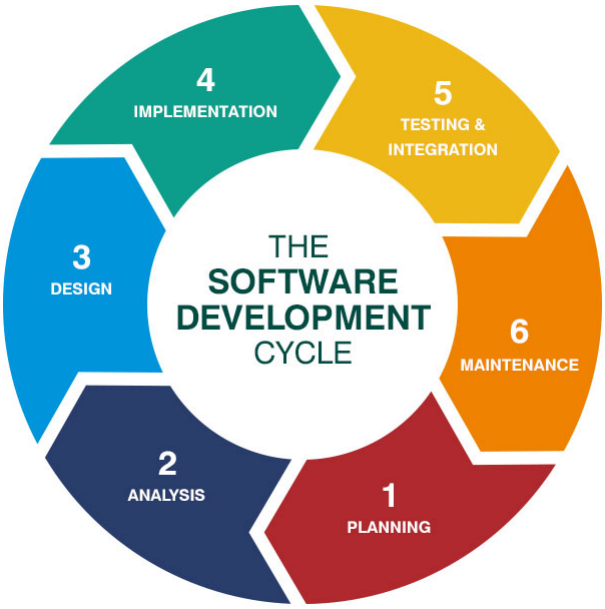



**Figure 1.2:** Software Development Life Cycle (SDLC) Phases [**sdlc-phases**]

### 1.6.2 Comparision of SDLC Models

There are several SDLC models that development teams can use, each with its own advantages and disadvantages. The most common SDLC models include:

**Table 1.1:** Comparison of SDLC Models

| **Model** | **Advantages** | **Disadvantages** |
|---|---|---|
| Waterfall Model | - Simple and easy to understand<br>- Well-suited for small projects<br>- Clear requirements and deliverables | - Inflexible to changes<br>- High risk of project failure<br>- Limited customer involvement |
| Agile Model | - Flexibility and adaptability<br>- Customer collaboration<br>- Incremental and iterative development | - Requires experienced team members<br>- Lack of documentation<br>- Difficult to predict project timeline |
| Scrum Model | - Lightweight and easy to implement<br>- Emphasizes teamwork and collaboration<br>- Iterative and incremental development | - Requires dedicated Scrum Master<br>- Can be challenging for large teams<br>- May not work well for fixed-price projects |
| Kanban Model | - Visual workflow management<br>- Limits work in progress<br>- Maximizes efficiency and productivity | - Lack of structure and planning<br>- Limited focus on project goals<br>- May not work well for complex projects |

and the list goes on. Choosing the right SDLC model depends on the project requirements, team size, and organizational culture.

### 1.6.3 Agile Methodology

Agile is an iterative and incremental approach to software development that emphasizes flexibility, collaboration, and customer feedback. Agile methodologies are based on the Agile Manifesto, which values:

- **Individuals and interactions** over processes and tools
- **Working software** over comprehensive documentation

- **Customer collaboration** over contract negotiation
- **Responding to change** over following a plan

While there is value in the items on the right, we value the items on the left more.

### 1.6.4 Scrum Framework

Scrum is a lightweight framework that helps development teams deliver high-quality software products by emphasizing teamwork, accountability, and iterative progress. The Scrum framework consists of several roles, events, and artifacts that work together to facilitate the development process, making it suitable for small to medium-sized teams working on complex projects. Scrum's approach fosters an environment where these advantages can be realized, leading to successful project outcomes.

**Reference:** For an in-depth understanding of Scrum, refer to the Scrum Guide [**scrumGuides**].

Figure **??** represents the scrum methodology cycle.

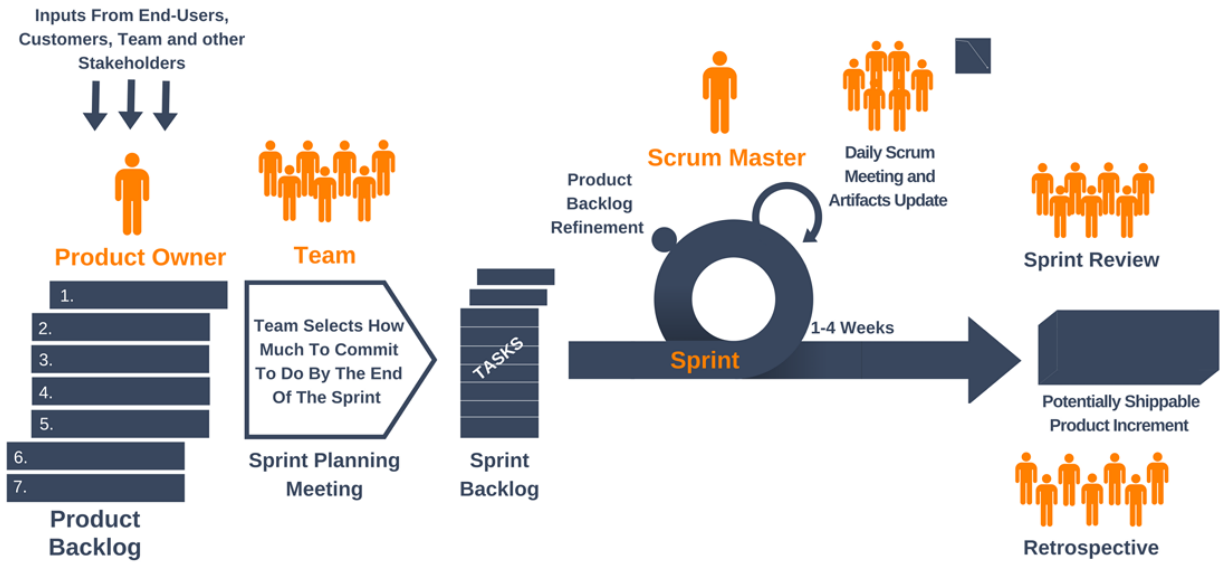



**Figure 1.3:** Illustration of the Scrum Framework [**scrum-framework**]

### 1.6.5 Scrum Roles

The Scrum framework defines three primary roles that are essential for the success of the project:

- **Product Owner** - Represents the stakeholders and is responsible for defining the product vision, prioritizing the backlog, and ensuring the team delivers value to the customer.
- **Scrum Master** - Facilitates the Scrum process, removes impediments, and helps the team work together effectively.
- **Development Team** - A self-organizing, cross-functional team responsible for delivering a potentially shippable product increment at the end of each sprint.

### 1.6.6 Scrum Events

The Scrum framework defines several events that occur within a sprint to facilitate communication, collaboration, and progress:

- **Sprint Planning** - A meeting where the team plans the work to be done in the sprint.
- **Daily Standup** - A brief daily meeting where the team synchronizes activities and plans for the day.
- **Sprint Review** - A meeting where the team demonstrates the work completed during the sprint and gathers feedback.
- **Sprint Retrospective** - A meeting where the team reflects on the sprint and identifies opportunities for improvement.

### 1.6.7 Scrum Artifacts

The Scrum framework defines several artifacts that provide transparency and visibility into the project's progress:

- **Product Backlog** - A prioritized list of features, enhancements, and bug fixes that need to be implemented. It is a dynamic and evolving artifact that serves as the single source of requirements for any changes to be made to the product. The Product Owner is responsible for managing and prioritizing the Product Backlog.

- **Sprint Backlog** - A subset of the Product Backlog items selected for the current sprint, along with a plan for delivering the product Increment and achieving the sprint goal. The development team is responsible for updating the Sprint Backlog throughout the sprint to reflect the progress of their work.

- **Increment** - The sum of all the Product Backlog items completed during a sprint and all previous sprints. The Increment must be in a usable condition and meet the Scrum Team's definition of "Done". It represents a concrete step towards the final product, providing value to the stakeholders.

### 1.6.8 Scrum Implementation

The Scrum framework will be implemented throughout the development process to ensure that the project stays on track and delivers value to the stakeholders. The team will follow the Scrum events, roles, and artifacts to facilitate communication, collaboration, and progress. Regular sprint planning, daily standups, sprint reviews, and sprint retrospectives will be conducted to keep the team aligned and focused on the project goals.

### 1.6.9 Scrum Tools

To support the Scrum framework, the team will use various tools to manage the project, communicate effectively, and track progress. These tools include:

- **GitHub** - A code repository that allows the team to collaborate on code, track changes, and manage versions.

- **Slack** - A messaging platform that enables real-time communication between team members, stakeholders, and clients.

- **Google Meet** - A video conferencing tool that facilitates virtual meetings, standups, and sprint reviews.
- **Notion** - A project management tool that provides a centralized platform for documentation, task management, and collaboration.

## 1.7 Project Timeline

During the internship, the project timeline was divided into several sprints, each lasting one month. The sprints were organized according to the Scrum framework, with sprint planning, daily standups, sprint reviews, and sprint retrospectives conducted at regular intervals to track progress and adjust the project scope as needed.

### 1.7.1 Product Owner & Scrum Master

The Product Owner and Scrum Master here is Sami Kammoun.

### 1.7.2 Product Backlog

**Table 1.3:** Product Backlog

| Sprint | User Story | ID | Task |
|---|---|---|---|
| 1 | 1.1 | 1.1.1 | As a user, I want to sign up using my email and password. |
| | | 1.1.2 | As a user, I want to sign in using my email and password. |
| | | 1.1.3 | As a user, I want to create a store during the sign-up process. |
| | | 1.1.4 | As a user, I want to update my store information or my user information. |
| | | 1.1.5 | As a user, I want the system to handle user and store CRUD operations gracefully. |

Continued on next page

**Table 1.3 – Continued from previous page**

| Sprint | User Story | ID | Task |
|---|---|---|---|
| | | 1.1.6 | As a user, I want the system to revalidate my JWT token automatically. |
| 2 | 2.1 | 2.1.1 | As a user, I want to create, read, update, and delete products. |
| | | 2.1.2 | As a user, I want to create, read, update, and delete categories. |
| | | 2.1.3 | As a user, I want to search for products by category. |
| | | 2.1.4 | As a user, I want to search for products by name. |
| | | 2.1.5 | As a user, I want to link products to categories. |
| | | 2.1.6 | As a user, I want to upload images for products and categories. |
| | | 2.1.7 | As a user, I want the system to handle product and category CRUD operations gracefully. |
| | | 2.1.8 | As a user, I want the system to perform searches and linking operations quickly. |
| | | 2.1.9 | As a user, I want the system to upload and serve images efficiently. |
| 3 | 3.1 | 3.1.1 | As a user, I want to create, read, update, and delete orders. |
| | | 3.1.2 | As a user, I want to search for orders by multiple fields. |
| | | 3.1.3 | As a user, I want to link orders to delivery services. |
| | | 3.1.4 | As a user, I want the system to handle order CRUD operations gracefully. |
| | | 3.1.5 | As a user, I want the system to perform order searches quickly. |
| | | 3.1.6 | As a user, I want the system to link orders to delivery services quickly. |

Continued on next page

**Table 1.3 – Continued from previous page**

| Sprint | User Story | ID | Task |
|---|---|---|---|
| 4 | 4.1 | 4.1.1 | As a user, I want to create, read, update, and delete users in my team. |
| | | 4.1.2 | As a user, I want to give specific permissions to team members. |
| | | 4.1.3 | As a user, I want the system to handle team CRUD operations gracefully. |
| | | 4.1.4 | As a user, I want the system to allow routes based on user permissions. |
| | | 4.1.5 | As a user, I want the system to revoke user permissions quickly. |
| 5 | 5.1 | 5.1.1 | As a user, I want to see changes made by other users in real-time. |
| | | 5.1.2 | As a user, I want to see changes made by other users in the same store. |
| | | 5.1.3 | As a user, I want the system to handle the real-time feature gracefully. |
| | | 5.1.4 | As a user, I want the system to allow socket connections based on user permissions. |
| | | 5.1.5 | As a user, I want the system to revoke user permissions quickly. |
| 6 | 6.1 | 6.1.1 | As a user, I want to track my orders in real-time. |
| | | 6.1.2 | As a user, I want to see the status of my orders. |
| | | 6.1.3 | As a user, I want the system to handle the order tracking feature gracefully. |
| | | 6.1.4 | As a user, I want the order tracking feature to not affect system performance. |

Continued on next page

**Table 1.3 – Continued from previous page**

| Sprint | User Story | ID | Task |
|---|---|---|---|
| | | 6.1.5 | As a user, I want the system to update order status in real-time. |

### 1.7.3 Gantt Chart

We used a Gantt chart to visualize the project timeline and track progress throughout the internship. The Gantt chart provided a visual representation of the project milestones, tasks, and dependencies, allowing the team to plan and execute the project effectively. Figure ?? shows the Gantt chart for the project timeline.

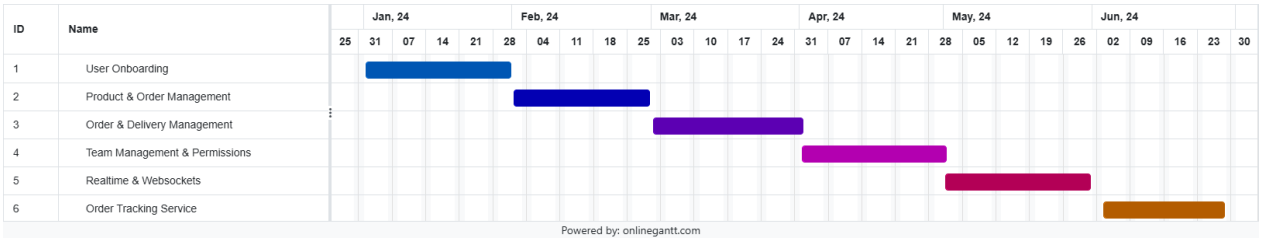



**Figure 1.4:** Project Gantt Chart

## 1.8 Conclusion

In this chapter, we outlined the scope of the project, identified the problems faced by SMEs in the e-commerce space, and defined the project goals and objectives. We also established a project management plan based on the Scrum framework and outlined the project timeline. In the next chapter, we will dive into the technical details of the project, including the architecture, design, and implementation of the e-commerce platform.

*Chapter 2*

# Project Setup

## 2.1 Introduction

This chapter outlines the project setup, including the tools and technologies used, the project's architecture, and the development environment. It provides an overview of the project's infrastructure, deployment, and cloud services to give a comprehensive understanding of the project's setup.

## 2.2 Common Practices - A Brief Comparison

### 2.2.1 Backend & Services

In this section, we will explore common practices in backend development and services. These practices ensure scalability, security, and efficiency in developing and maintaining backend systems.

#### 2.2.1.1 Backend Frameworks

Backend frameworks provide the necessary structure and tools for building and deploying server-side applications. Some common practices include:

- **Scalability and Performance**: Choosing frameworks that support asynchronous operations and non-blocking I/O to handle numerous concurrent connections efficiently.
- **Modularity**: Designing the backend with a modular architecture, enabling the separation of concerns and easier maintenance. This involves using middleware, services, and controllers.
- **Security**: Implementing robust security practices, including input validation, authentication, authorization, and protection against common vulnerabilities (e.g., SQL injection, XSS).
- **RESTful APIs**: Building APIs following REST principles to ensure statelessness, uniformity, and the use of standard HTTP methods for communication.
- **Microservices**: Adopting a microservices architecture to break down the application into smaller, independently deployable services that can be developed and scaled individually.

### 2.2.2 Frontend Technologies - SSR vs CSR

#### 2.2.2.1 Server-Side Rendering (SSR) vs Client-Side Rendering (CSR)

Frontend technologies play a crucial role in delivering a seamless user experience. Two common approaches are Server-Side Rendering (SSR) and Client-Side Rendering (CSR). Each approach has its advantages and use cases:

**Table 2.1:** Server-Side Rendering (SSR) vs Client-Side Rendering (CSR)

| Approach | Description & Use Case |
|---|---|
| **Server-Side Rendering (SSR)** | In SSR, the server generates the HTML content and sends it to the client, reducing the initial load time and improving SEO. SSR is suitable for content-heavy websites and applications that require fast initial rendering. |
| **Client-Side Rendering (CSR)** | In CSR, the client-side JavaScript renders the content dynamically, providing a more interactive and responsive user experience. CSR is ideal for single-page applications (SPAs) and dynamic web applications that require frequent updates and real-time interactions like React, Angular, and Vue.js. |
| **Hybrid Approaches** | Some frameworks, like Next.js, combine SSR and CSR to leverage the benefits of both approaches. This hybrid approach allows developers to pre-render pages at build time while enabling client-side navigation and dynamic content updates. |

#### 2.2.2.2 State Management

State management is essential for managing the application's data and UI state. Common state management libraries include Redux, MobX, and Context API. These libraries help manage complex state logic, share data across components, and ensure predictable state changes. Choosing the right state management solution depends on the application's

complexity, scalability requirements, and developer preferences.

### 2.2.3 Database Technologies - SQL vs NoSQL

Database technologies are essential for storing, managing, and retrieving data efficiently. Two common types of databases are SQL (relational) and NoSQL (non-relational).

The table below represents the main differences between SQL and NoSQL databases:

**Table 2.2:** SQL vs NoSQL Databases

| **Feature** | **SQL Databases** | **NoSQL Databases** |
|---|---|---|
| **Data Model** | Tabular (rows and columns) | Non-tabular (document, key-value, wide-column, graph) |
| **Schema** | Fixed schema with predefined structure | Dynamic schema with flexible structure |
| **Scalability** | Vertical scaling (increasing hardware resources) | Horizontal scaling (adding more servers) |
| **Query Language** | SQL (Structured Query Language) | Query language specific to the database (e.g., MongoDB Query Language) |
| **ACID Properties** | ACID (Atomicity, Consistency, Isolation, Durability) | BASE (Basically Available, Soft state, Eventually consistent) |
| **Use Cases** | Transactional applications, complex queries | Big data, real-time applications, unstructured data |
| **Examples** | MySQL, PostgreSQL, SQLite | MongoDB, Cassandra, Redis |

## 2.3 Converty's Approach - Used Technologies

### 2.3.1 Backend & Services

Since Converty follows a Modular Monolith architecture, the main backend which holds the authentication system, the simple CRUD operations, and the business logic is built using Node.js and Express.js. The microservices are built using Go (Golang) and are responsible for handling specific tasks like sending notifications, cron jobs, serverless functions, and other background tasks. This architecture allows for a flexible and scalable system that can handle different types of workloads efficiently.

#### 2.3.1.1 Node.js - Express.js

For the main backend framework, we chose Node.js with Express.js. Node.js is a JavaScript runtime built on Chrome's V8 engine, enabling server-side JavaScript development. Express.js is a minimalist web framework for Node.js, providing essential features for building APIs and web applications.



**Figure 2.1:** Node.js Logo

Node.js and Express.js offer several benefits, including:

- **Performance**: Node.js's event-driven, non-blocking I/O model allows for high performance and scalability, making it ideal for handling concurrent requests.
- **Modularity**: Express.js's middleware architecture enables modular design, allowing developers to add functionality through middleware functions.
- **Simplicity**: Node.js and Express.js are easy to learn and use, making them suitable for rapid development and prototyping.

- **Community Support**: Node.js and Express.js have a large and active community, providing a wealth of resources, plugins, and libraries to extend functionality.

#### 2.3.1.2 Go (Golang)



**Figure 2.2:** Golang Logo

In addition to Node.js, we used Go (Golang) for building microservices and APIs. Go is a statically typed, compiled language known for its simplicity, performance, and concurrency support. Go's built-in concurrency primitives, like goroutines and channels, make it well-suited for building high-performance applications. We chose Go for its efficiency, scalability, and ease of deployment, making it an excellent choice for building microservices and APIs.

#### 2.3.1.3 Queues - RabbitMQ




**Figure 2.3:** RabbitMQ Logo

For handling asynchronous tasks and background processing, we used RabbitMQ, a message broker that supports message queuing and pub/sub messaging patterns. RabbitMQ provides reliable message delivery, fault tolerance, and scalability, making it ideal for decoupling components and handling distributed systems. We used RabbitMQ to manage tasks, events, and notifications in the system, ensuring reliable and efficient communication between services.

#### 2.3.1.4 Authentication - JWT



**Figure 2.4:** JWT Logo

For authentication, we used JSON Web Tokens (JWT) to secure the application and authenticate users. JWT is an open standard for securely transmitting information between parties as a JSON object. JWT tokens are signed and encrypted, ensuring data integrity and authenticity. We used JWT tokens to authenticate users, authorize access to resources, and maintain user sessions securely. JWT tokens are stateless, scalable, and secure, making them ideal for modern web applications.

### 2.3.2 CSR - React




**Figure 2.5:** React Logo

For the frontend, we chose React as our primary library for building user interfaces. React is a JavaScript library developed by Facebook that allows developers to create interactive user interfaces efficiently. It uses a component-based architecture, where each component manages its state and can be reused across the application. React's virtual DOM (Document Object Model) allows for efficient updates to the UI, improving performance and user experience. In comparison to the MVC (Model-View-Controller) architecture, React's component-based approach simplifies the development process and makes it easier to reason about the application's state. Which provides a more predictable and maintainable codebase.

In addition to React, we used TypeScript for static typing and Vite for fast development and optimized builds. TypeScript adds static type checking to JavaScript, helping catch errors early and improve code quality. Vite is a modern build tool that provides fast development and optimized builds for React applications. Together, React, TypeScript, and Vite offer a powerful and efficient development environment for building responsive and interactive user interfaces.

#### 2.3.2.1 State Management - Redux




**Figure 2.6:** Redux Logo

For state management, we used Redux, a predictable state container for JavaScript applications. Redux helps manage the application's state in a single store, following a unidirectional data flow. Actions are dispatched to update the state, and components can subscribe to changes in the state. Redux simplifies the management of complex state logic, making it easier to debug and test the application. Redux's centralized state management ensures consistency and predictability in the application's state changes.

#### 2.3.2.2 Styling - CSS and Styled-components

For styling, we created a design system using simple CSS with utility classes and styled-components for component styling. Styled-components allow us to write CSS directly in our JavaScript files, making it easier to manage styles and keep them in sync with the components. This approach ensures consistency and reusability across the application, making it easier to maintain and update the design system.

### 2.3.3 NoSQL - MongoDB




**Figure 2.7:** MongoDB Logo

For the database, we chose MongoDB as our primary database technology. MongoDB is a NoSQL database that stores data in flexible, JSON-like documents. It provides a scalable, high-performance, and flexible data storage solution that is well-suited for modern applications. MongoDB's document-based data model allows for easy storage and retrieval of complex data structures, making it ideal for applications with dynamic schemas. We chose MongoDB for its scalability, performance, and ease of use, making it an excellent choice for storing and managing data in our application.

#### 2.3.3.1 Data Modeling - Mongoose

For data modeling, we used Mongoose, an Object Data Modeling (ODM) library for MongoDB. Mongoose provides a simple and elegant way to model data in MongoDB, allowing developers to define schemas, validate data, and perform CRUD operations easily. Mongoose's schema-based approach makes it easy to define data models and relationships, ensuring consistency and integrity in the database.

#### 2.3.3.2 Caching - Redis




**Figure 2.8:** Redis Logo

We used Redis as a caching layer to improve performance and reduce latency. Redis is an open-source, in-memory data store that supports various data structures like strings, lists, sets, and hashes. It provides fast read and write operations, making it ideal for caching frequently accessed data. Redis helps reduce the load on the database and improves the

application's responsiveness by storing frequently accessed data in memory. In addition to caching, Redis can be used for session management, pub/sub messaging, and other use cases that require fast data access.

### 2.3.4 Cloud Services - GKE (Google Kubernetes Engine)

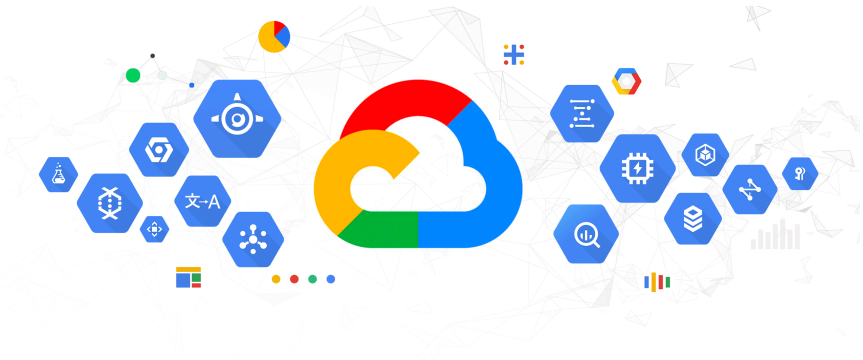


**Figure 2.9:** GKE Logo

For deployment, we migrated the application to GKE (Google Kubernetes Engine), a managed Kubernetes service that provides a scalable, reliable, and secure environment for running containerized applications. GKE offers a robust set of features for managing Kubernetes clusters, including auto-scaling, load balancing, and monitoring. We used GKE to deploy our application in a production environment, ensuring high availability, scalability, and reliability.

## 2.4 Validation

### 2.4.1 Results

This choice of technologies helped in building:

- **A scalable and efficient** backend system using Node.js and Express.js.
- **A microservices architecture** using Go for specific tasks and background processes.
- **A responsive and interactive** frontend using React and Redux for state management.
- **A flexible and scalable** database using MongoDB and Mongoose for data modeling.

- **A caching layer** using Redis to improve performance and reduce latency.
- **A reliable and secure** deployment using GKE for running containerized applications.

### 2.4.2 Discussion

One of main challenges faced during this project was the communication and synchronization between the different components of the system. Integrating the frontend with the backend, microservices, and database required careful planning and coordination to ensure seamless operation.

## 2.5 Summary & Conclusion

In this chapter, we discussed the project setup, including the tools and technologies used, the project's architecture, and the development environment. We explored common practices in backend development, frontend technologies, database technologies, and cloud services. We compared Converty's approach to common practices and discussed the results and challenges faced during the project setup. In the next chapters, we will discuss every module in detail and how it was implemented and validated.

*Chapter 3*

# User Onboarding

## 3.1 Introduction

In this chapter, we will discuss the user onboarding process for the system. The user onboarding process is the process of getting a new user to sign up and start using the system. We will start by discussing the signup logic then login logic

## 3.2 Sprint Backlog

**Table 3.1:** Sprint 1 backlog for User Onboarding

| Sprint | User Story | ID | Task |
|---|---|---|---|
| 1 | 1.1 | 1.1.1 | Backend: Implement user sign up API endpoint. |
| | | 1.1.2 | Frontend: Design and implement user sign up form. |
| | 1.2 | 1.2.1 | Backend: Implement user sign in API endpoint. |
| | | 1.2.2 | Frontend: Design and implement user sign in form. |
| | 1.3 | 1.3.1 | Backend: Modify user sign up to include store creation. |
| | | 1.3.2 | Frontend: Update sign up form to include store information. |
| | 1.4 | 1.4.1 | Backend: Implement API endpoints for updating user and store information. |
| | | 1.4.2 | Frontend: Develop forms for updating user and store information. |

## 3.3 Design & Implementation

In this section, we will discuss the design and implementation of the user onboarding process. We will start by discussing the backend design and implementation then move on to the frontend design.

### 3.3.1 Backend

#### 3.3.1.1 Database Schema

Let's start by discussing the database schema for the system. The database schema for the system is shown in Figure ??.

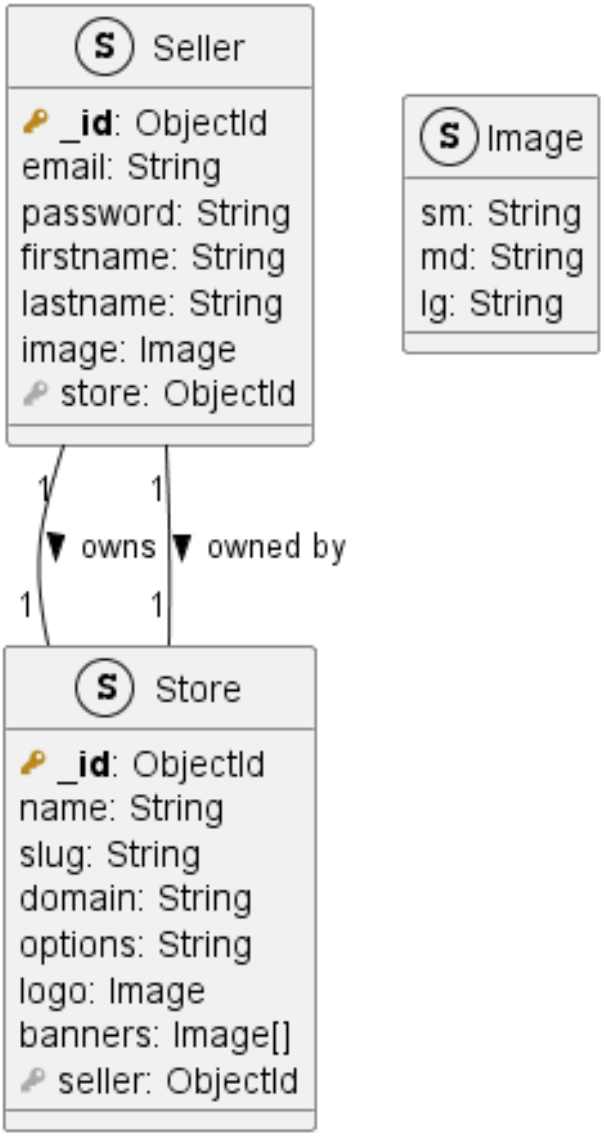



**Figure 3.1:** Store and User Database Schema

In this database schema, we have two schemas, the user schema, and the store schema. The user schema contains the user's email, password, and role. The store schema contains the store's name and other information. Each entity is linked to the other entity using an ID field. and this helps finding and loading the store info quicker at the time of login and

locking the store to one user for security reasons.

#### 3.3.1.2 Authentication - JWT

**Design Logic**

- **Signup** - When a user signs up, the system will create a user and a store for the user. The system will then create a JWT token for the user and send it back to the user. We can use the figure **??** to show the sequence of the signup process.

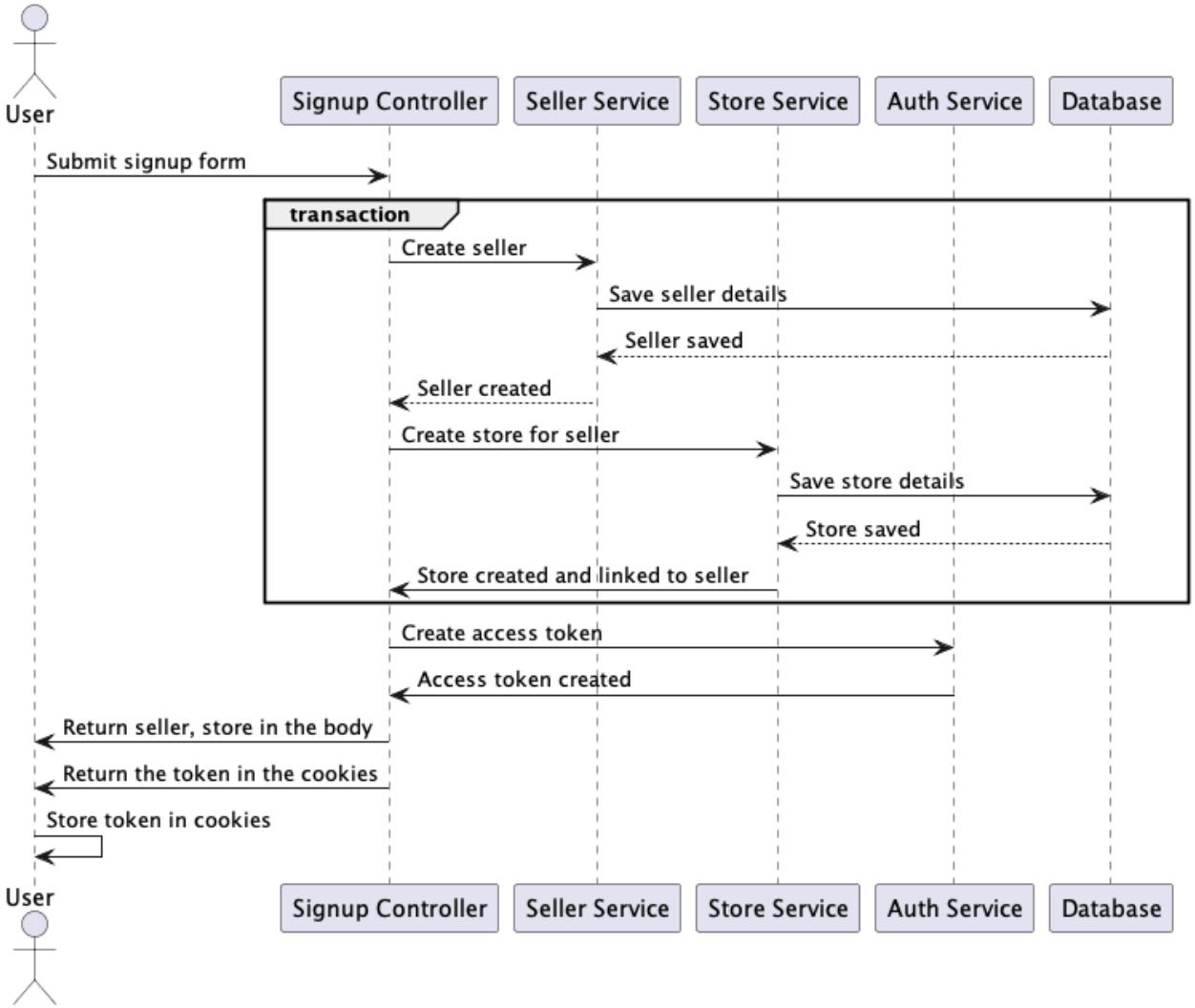



**Figure 3.2:** Signup Sequence Diagram

- **Login** - When a user logs in, the system will validate the user's email and password. If the user's email and password are correct, the system will create a JWT token for the user and send it back to the user. We can use the figure **??** to show the sequence

of the login process.

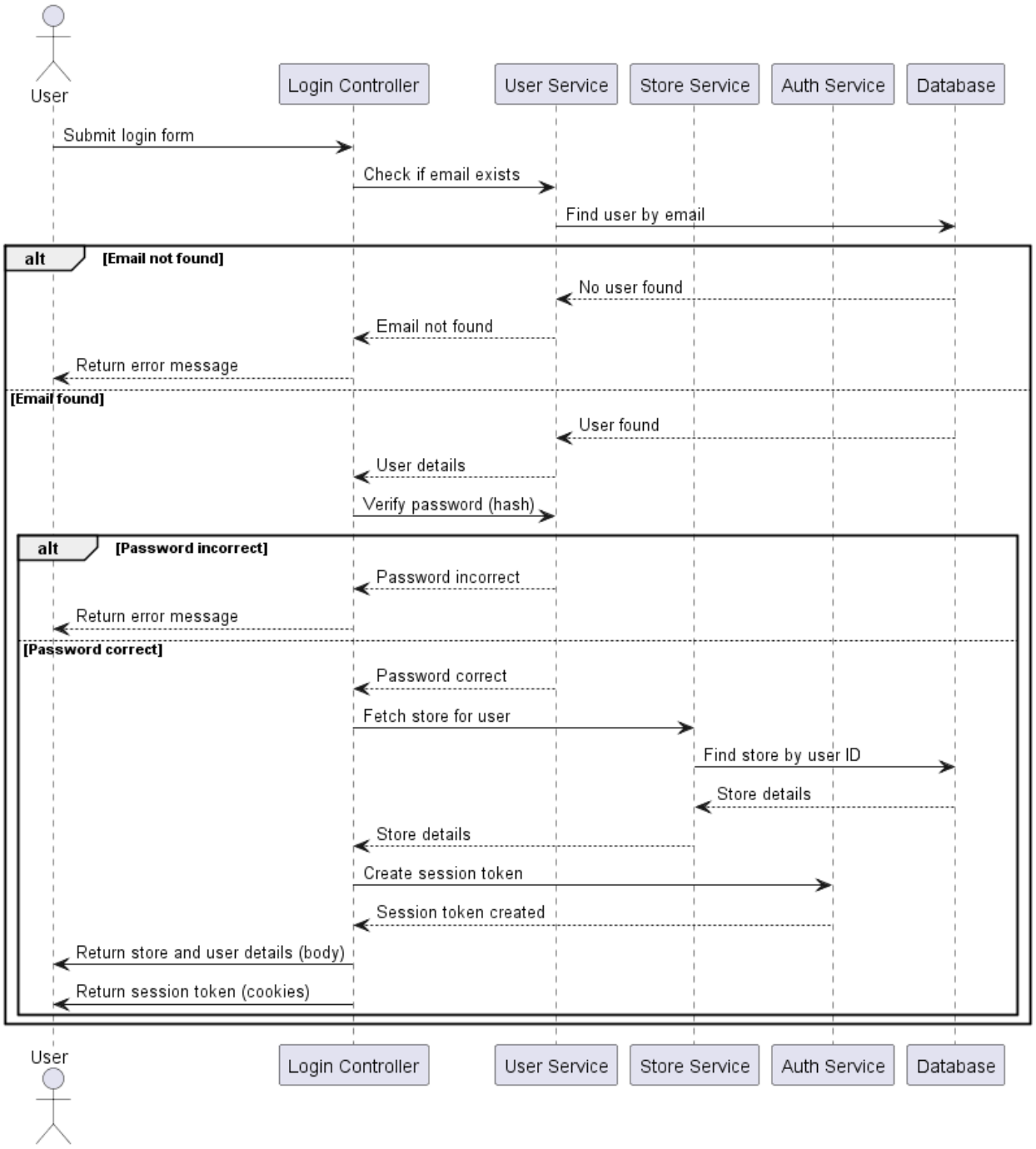



**Figure 3.3:** Login Sequence Diagram

**Implementation** The implementation of the JWT token is done using the `jsonwebtoken` library in Node.js. The main benfits of using JWT tokens are: [**jwtBenefits**]

- JWT tokens are stateless, which means that the server does not need to store the user's session information.

- JWT tokens are secure, which means that the server can verify the token's signature to ensure that the token has not been tampered with.
- JWT tokens are scalable, which means that the server can verify the token's signature without having to query the database.

We can use the code snippet ?? to show how the JWT token is implemented in the system.

```
const jwt = require('jsonwebtoken');

const createToken = (payload) => {
    return jwt.sign(payload, precess.env.SECRET, { expiresIn: "2d" });
};
```

Listing 3.1: JWT Token Implementation

Moreover, we can see the payload we stored in the JWT token in this figure ??.

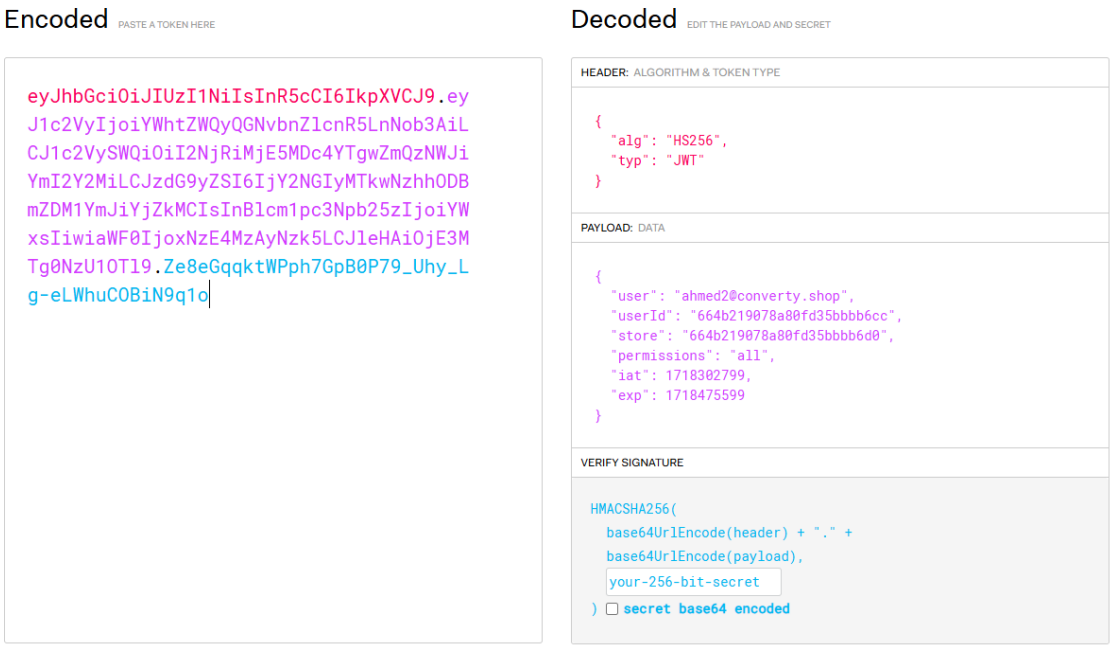



**Figure 3.4:** JWT Token Payload

We stored the userId, email, permissions and store id in the JWT token payload to ease up the authentication process when accessing APIs that require authentication and to provide the required information easily.

#### 3.3.1.3 APIs - User & Store CRUD

**User CRUD**

- **Read User** - The system should allow users to read their user information.
- **Update User** - The system should allow users to update their user information.

We skipped the simple api for creating a user since it is part of the signup process and the delete user api since it requires a more complex logic to handle the store deletion and the user deletion and all the related data.

**Store CRUD**

- **Read Store** - The system should allow users to read their store information.
- **Update Store** - The system should allow users to update their store information.

As you may have noticed before we don't have the logic for creating a store since it is part of the signup process and the delete store api since it requires the user to delete the store before the user is deleted.

**API Diagram** We can use the sequence diagram **??** to show the API diagram for the user and store CRUD operations. We will take for example the logic of updating the store information.

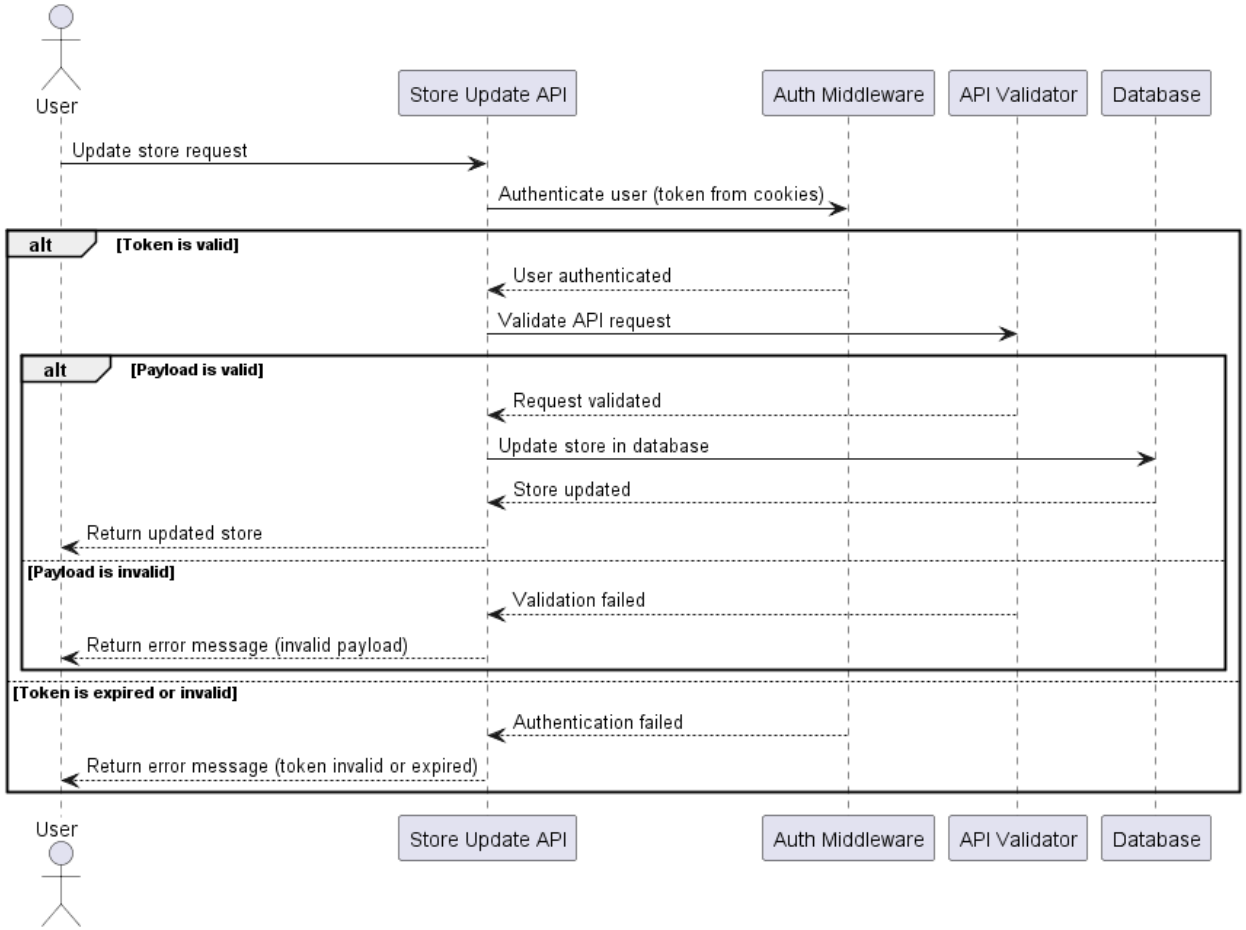



**Figure 3.5:** API Sequence Diagram

### 3.3.2 Frontend

#### 3.3.2.1 User Journey

We recreated the user journey for the user onboarding process in the system. We can use the figure **??** to show the user journey for the user onboarding process.

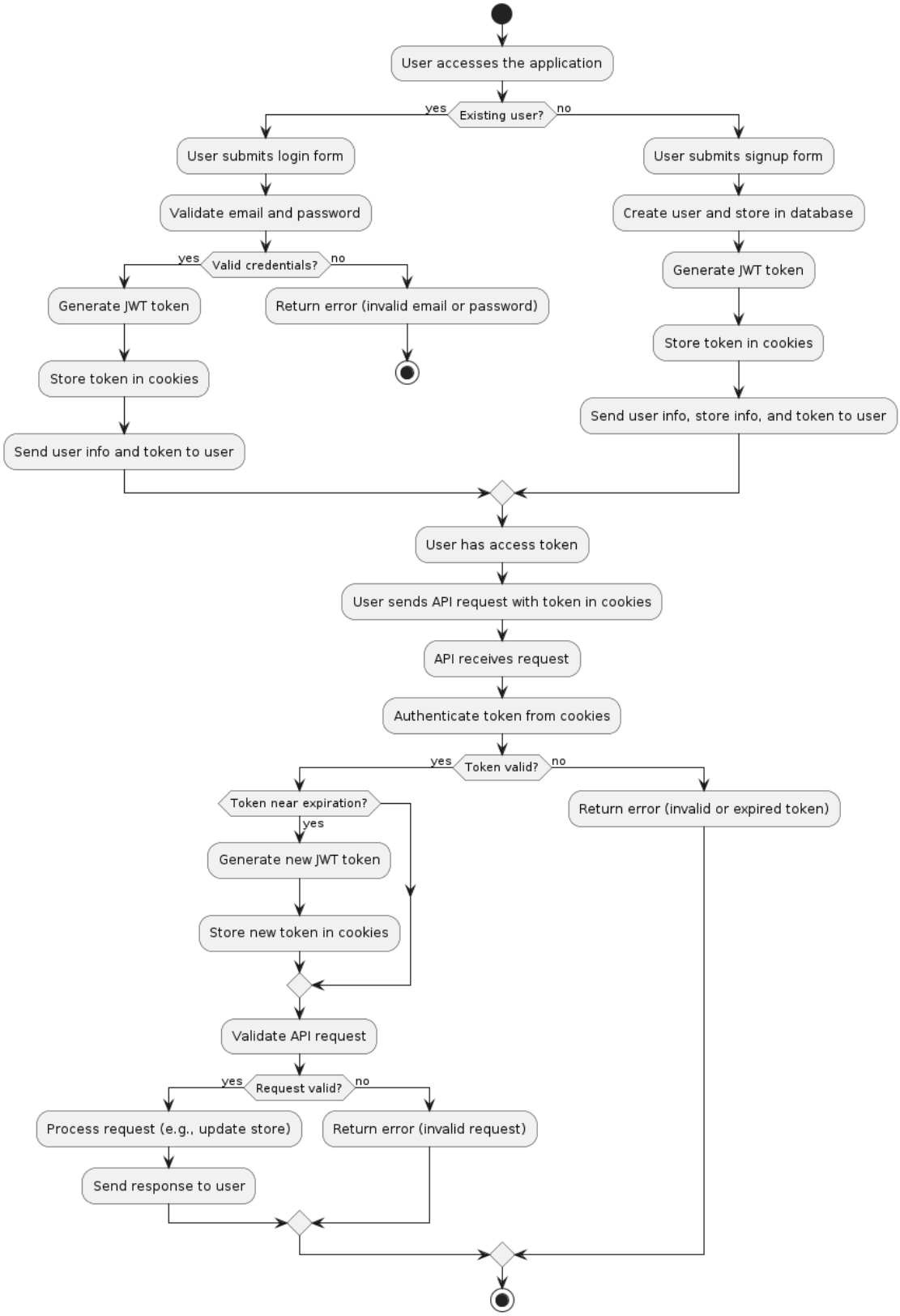



**Figure 3.6:** User Journey

#### 3.3.2.2 UI Design

**Signup UI** This is the UI for the signup page. The user should fill the form with their email, password, phone, store name and password. We have an invitation code field too since the platform is not public and we need to invite users to use the platform. Figure **??** shows the signup UI.

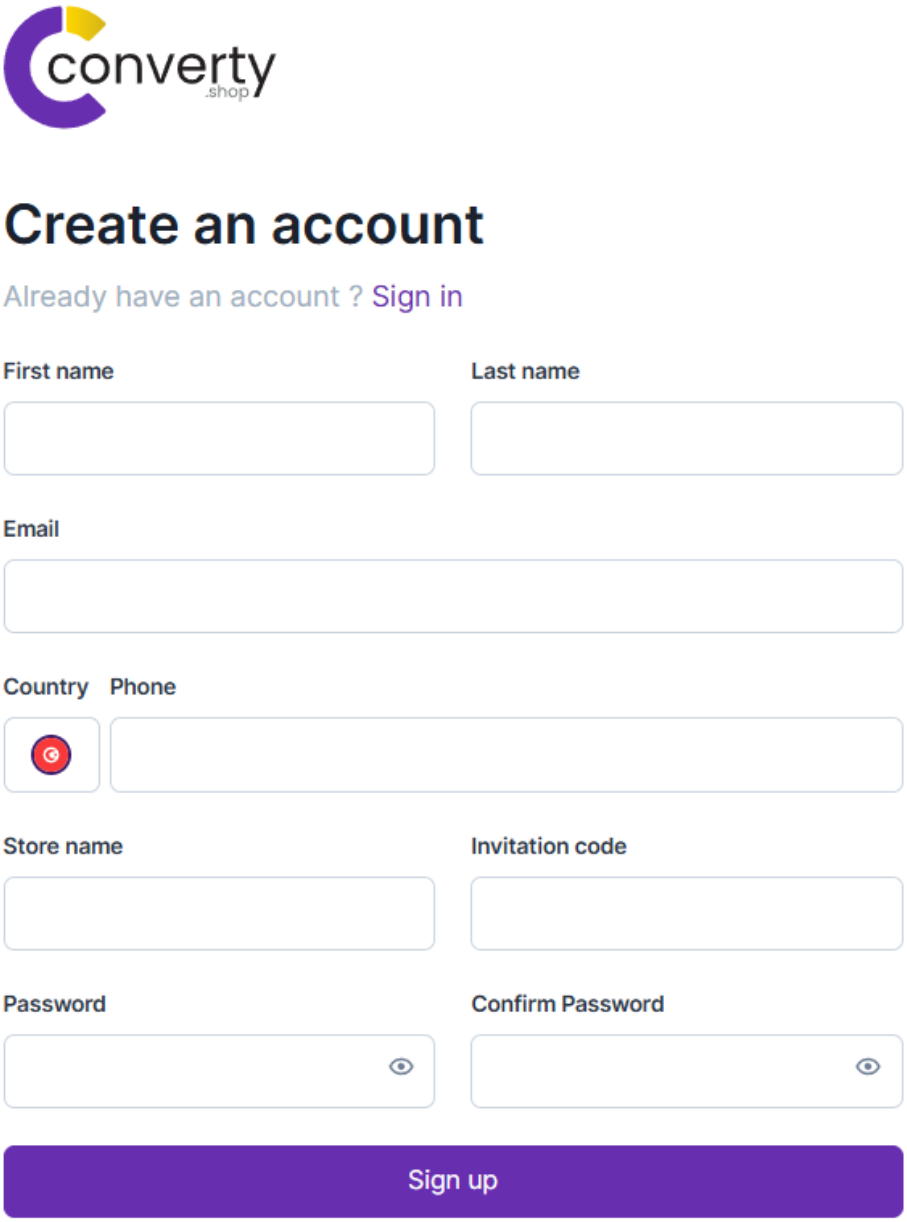



**Figure 3.7:** Signup UI

**Login UI** In the login page, the user should fill the form with their email and password. Figure ?? shows the login UI.

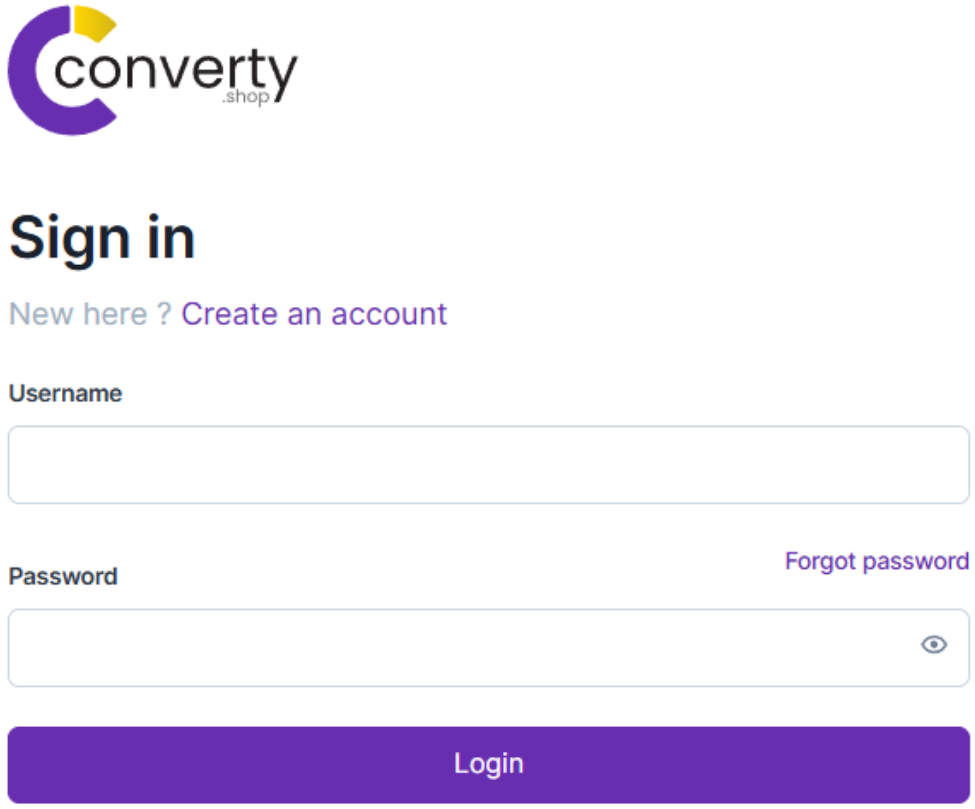



**Figure 3.8:** Login UI

**Profile Page UI** In the profile page, the user here can update their information as figure ?? shows.

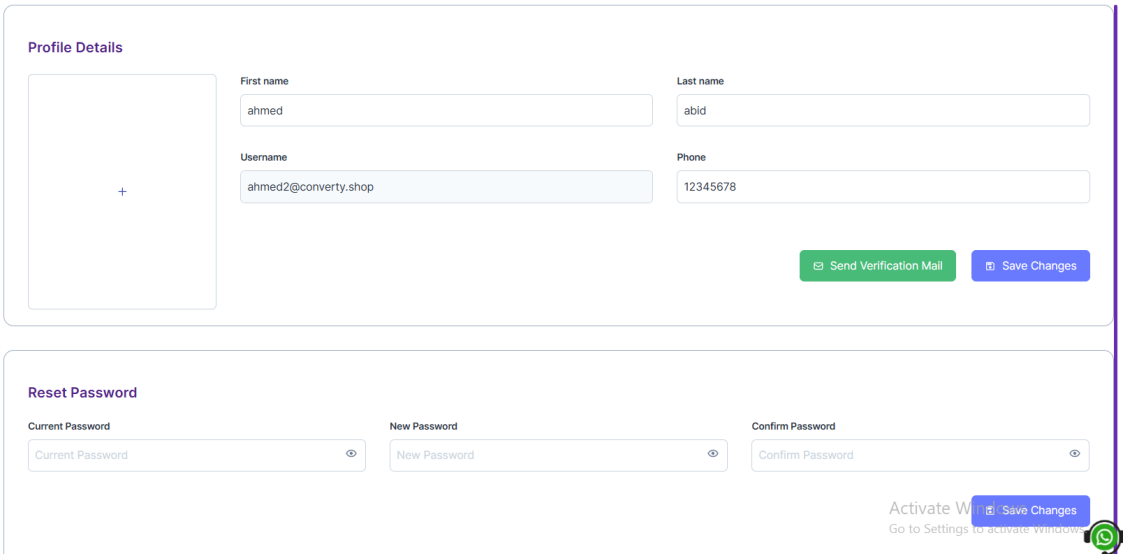



**Figure 3.9:** Profile Page UI

## 3.4 Validation

In this section we will critisize the system and discuss the results of the user onboarding process.

### 3.4.1 Results

One of the main results of the user onboarding process is that the users were able to sign up and create a store within 5 minutes. This is a good result since it shows that the system is easy to use and user-friendly. Moreover, the fluid authentication process helped the users to maintain their sessions for the longest time and ease up the process of using the system endpoints through the sleak interface.

### 3.4.2 Discussion

One of the main issues with the user onboarding process is that the reset password feature was not implemented correctly. The reset password feature should invalidate old/given JWT tokens created by the system earlier and create a new one for the user. This is a security issue since the user can still use the old JWT token to access the system even after resetting the password. We can not invalidate the old tokens since we are using a stateless

authentication system and the user can still use the old token to access the system. We can use the figure ?? to show the sequence of the reset password process.

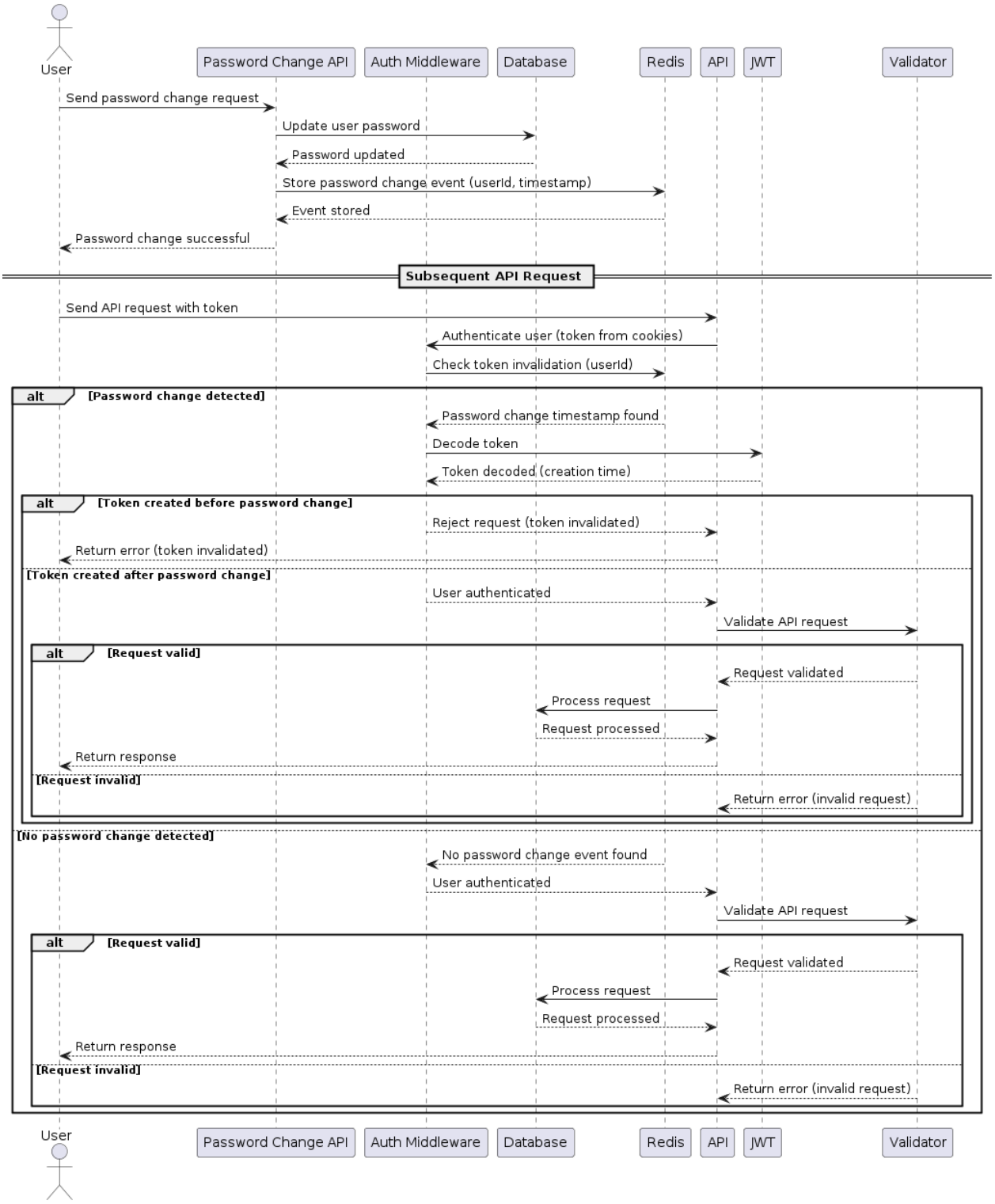



**Figure 3.10:** Reset Password Sequence Diagram

## 3.5 Summary & Conclusion

In this chapter we discussed the user onboarding process for the system. We started by discussing the requirements for the user onboarding process. We then discussed the design and implementation of the user onboarding process. We discussed the backend design and implementation, the frontend design and implementation, and the validation of the system. We then discussed the results of the user onboarding process and the issues with the user onboarding process. We concluded by discussing the user onboarding process and the results of the user onboarding process.

*Chapter 4*

# Product and Category Management

## 4.1 Introduction

In this chapter we will discuss the product and category management process for the system. The product and category management process is the process of managing the products and categories in the system. We will start by discussing the product and category CRUD operations then move on to the product and category search operations.

## 4.2 Sprint Backlog

**Table 4.1:** Sprint 2 backlog for Product & Category Management

| Sprint | User Story | ID | Task |
|---|---|---|---|
| 2 | 2.1 | 2.1.1 | Backend: Implement CRUD API endpoints for products. |
| | | 2.1.2 | Frontend: Design product management UI for CRUD operations. |
| | 2.2 | 2.2.1 | Backend: Implement CRUD API endpoints for categories. |
| | | 2.2.2 | Frontend: Design category management UI for CRUD operations. |
| | 2.3 | 2.3.1 | Backend: Implement API endpoints for searching products by category. |
| | | 2.3.2 | Frontend: Implement product search functionality by category. |
| | 2.4 | 2.4.1 | Backend: Implement API endpoints for searching products by name. |
| | | 2.4.2 | Frontend: Implement product search functionality by name. |
| | 2.5 | 2.5.1 | Backend: Implement functionality to link products to categories. |
| | 2.6 | 2.6.1 | Frontend: Develop UI for linking products to categories. |

Continued on next page

**Table 4.1 – Continued from previous page**

| Sprint | User Story | ID | Task |
|---|---|---|---|
| | 2.7 | 2.7.1 | Backend: Implement API endpoints for uploading images for products and categories. |
| | | 2.7.2 | Frontend: Implement image upload functionality for products and categories. |

## 4.3 Design & Implementation

In this section we will talk about the design and implementation of the product and category management process. We will start by discussing the backend design and implementation then move on to the frontend design and implementation.

### 4.3.1 Backend

#### 4.3.1.1 Database Schema

Let's start by discussing the database schema for the system. The database schema for the system is shown in Figure **??**.

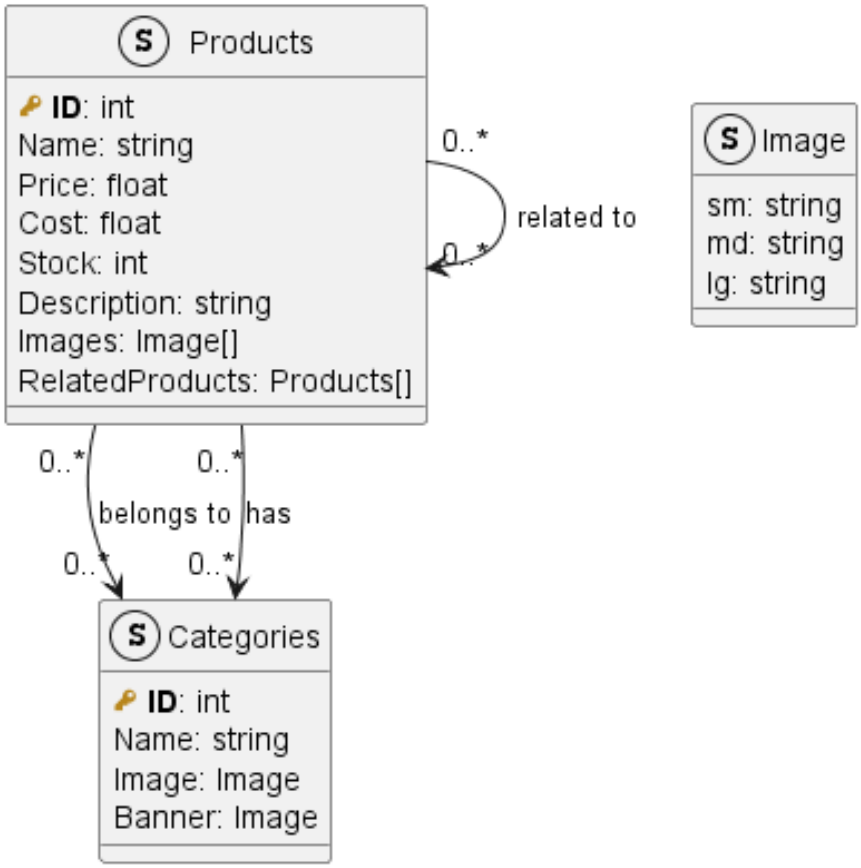



**Figure 4.1:** Product and Category Database Schema

#### 4.3.1.2 Search Operations

**Design Logic** In this paragraph, we will discuss the design logic behind the search operations for the products and categories. The search operations are the operations that allow users to search for products by category and name. The search operations are shown in figure **??**.

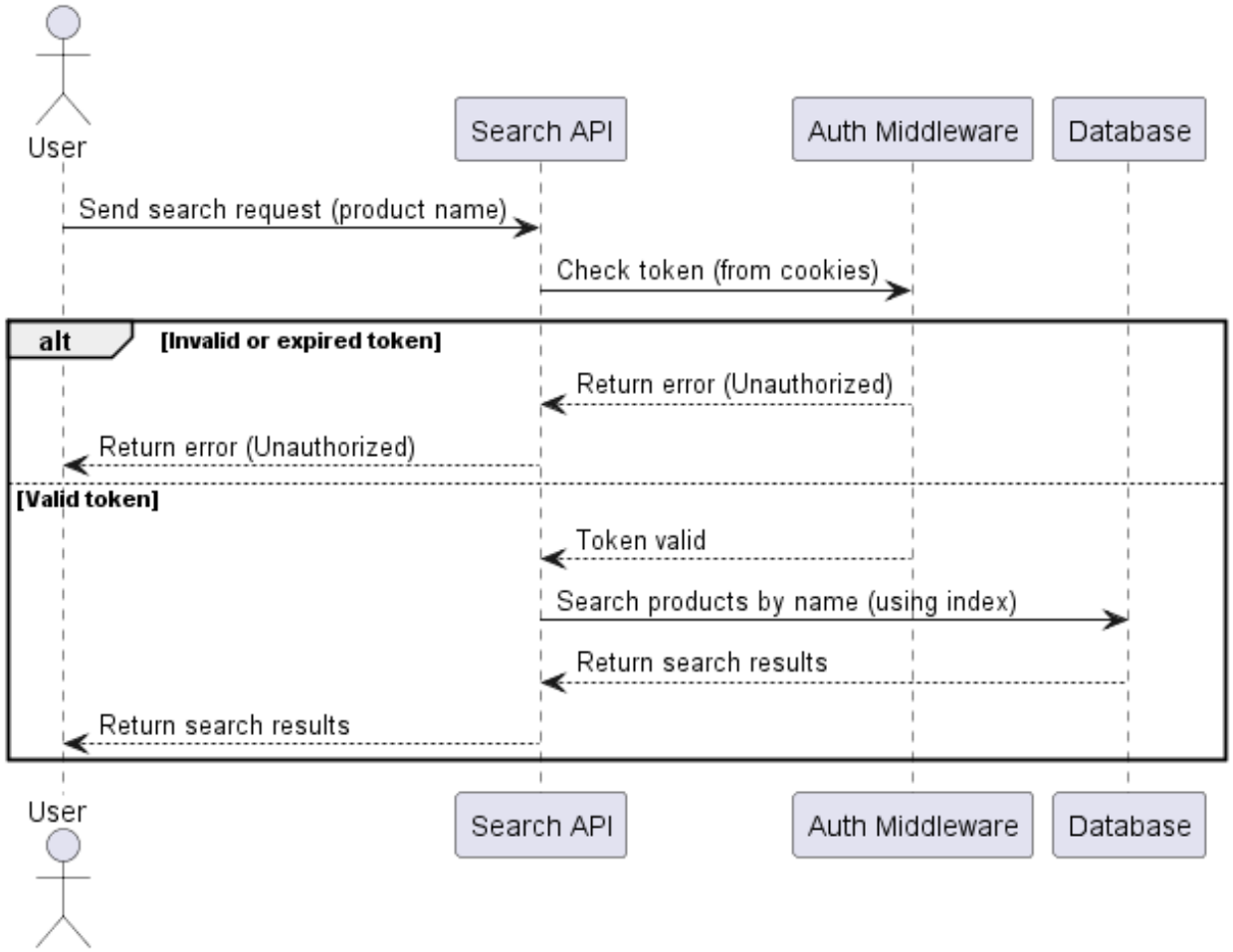



**Figure 4.2:** Search Operations Sequence Diagram

**Implementation** The implementation part is really simple, we just need to index the fields that we are going to search by. The tricky part here is to choose the right fields to index like we did in the database schema.

We use indexes usually to speed up the search/fetch operations for the frequently used fields in the system.

#### 4.3.1.3 Product & Category CRUD

**Design Logic** In this paragraph we will discuss the design logic for the product and category CRUD operations. The product and category CRUD operations are the operations that allow users to create, read, update, and delete products and categories. We use figure **??** to show the sequence of the product and category CRUD operations. (let's assume that the category is created and we will just assign it to the product)

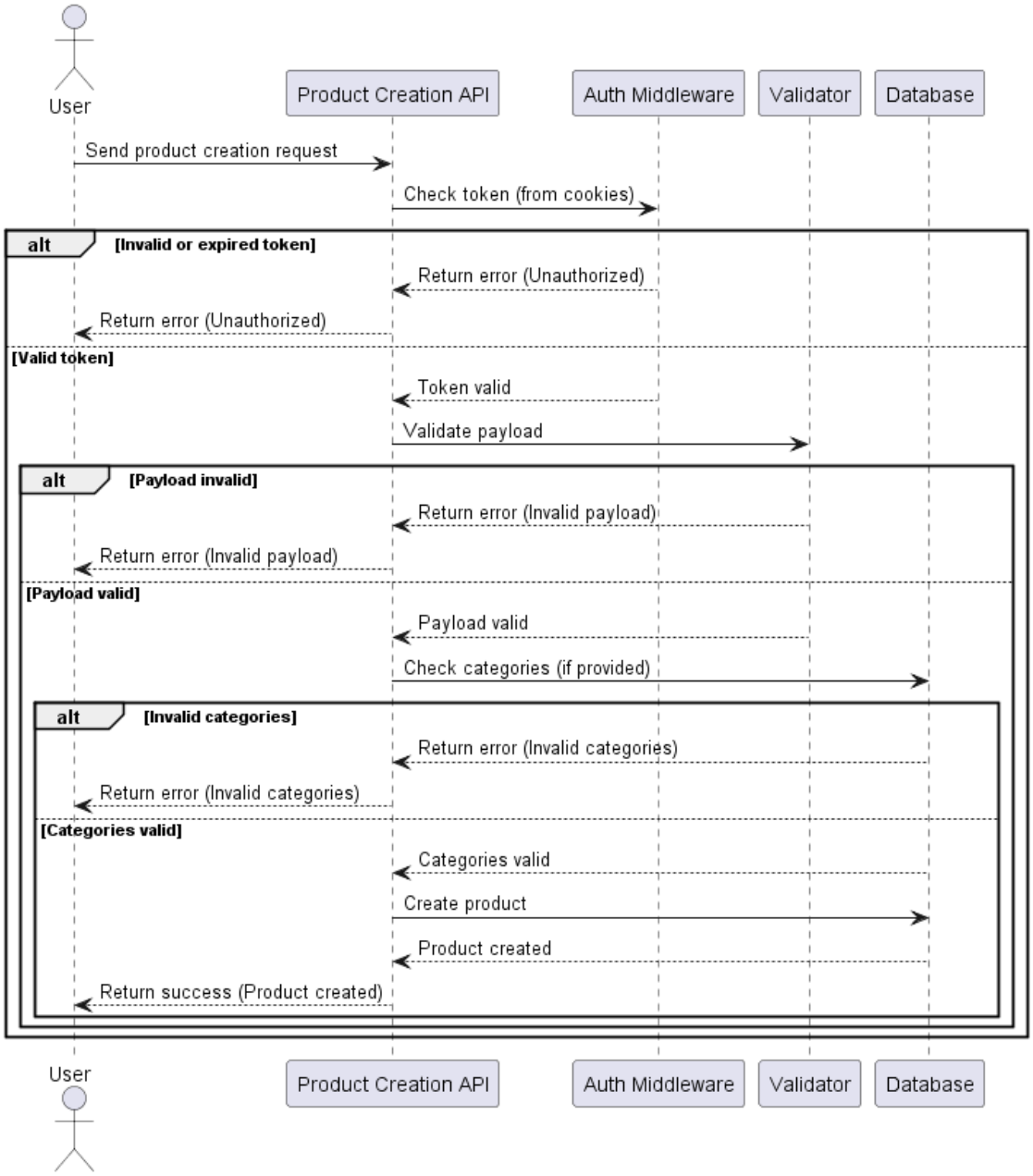



**Figure 4.3:** Product and Category CRUD Sequence Diagram

The same logic goes for the category CRUD operations. The system will allow users to create, read, update, and delete categories. The system will also allow users to link the products to the categories.

#### 4.3.1.4 Caching Layer

In this section we will discuss the caching layer for the system. The caching layer is used to cache the products and categories in the system. The caching layer is used to

speed up the search operations for the products and categories. We use figure ?? to show the sequence of the caching layer.

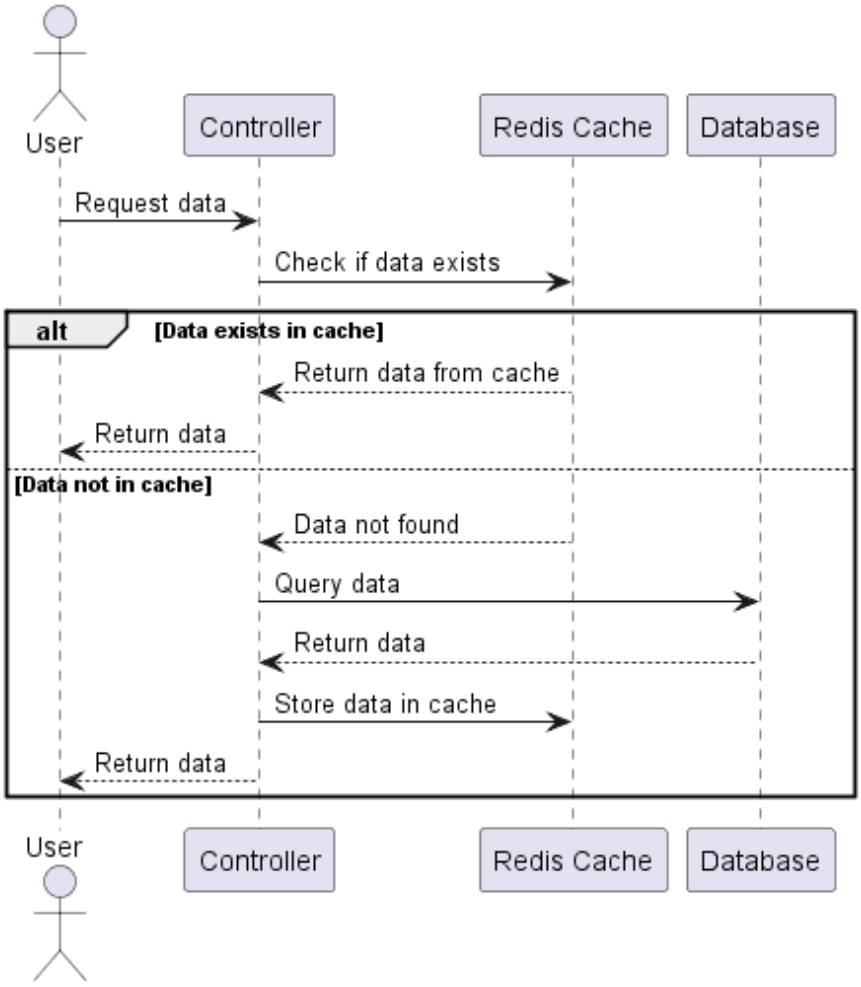



**Figure 4.4:** Caching Layer Sequence Diagram

Benefits of the caching layer:

- Speeds up the search operations for the products and categories.
- Reduces the load on the database.
- Reduces the load on the system.
- Limits the rate of requests for the main database.

and this is the code snippet of how the caching works in the system:

```
1    let data = await redisClient.get(key);
```

```
2
3     if (data) {
4         console.log('Data fetched from Redis');
5         return JSON.parse(data);
6     }
7
8     // Data not found in Redis, query MongoDB
9     const collection = db.collection('mycollection');
10    data = await collection.findOne({ key });
11
12    if (data) {
13        console.log('Data fetched from MongoDB');
14        redisClient.setex(key, 3600, JSON.stringify(data));
15    }
16
17    return data;
```

Listing 4.1: Caching Layer Implementation

#### 4.3.1.5 Image Upload

In this section we will discuss the image upload process for the system. But before we dive into the logic behind uploading and serving images in the system let's discuss few topics first. We know that the process of uploading images and serving them to the clients using our own server can be ressource consuming and slow. To solve this issue we will use the following services:

**S3 Bucket** The S3 bucket is a storage service provided by Amazon Web Services. The S3 bucket is used to store the images for the products and categories in the system. The S3 bucket is used to store the images in a secure and scalable way. The S3 bucket is used to store the images and provides the users with a link to access the images.

**AWS CDN** AWS Cloudfront is a content delivery network provided by Amazon Web Services. The AWS Cloudfront is used to cache the images and serve them to the users in

the best way possible. The AWS Cloudfront is used to cache the images and serve them to the users in the fastest way possible.

Moreover, AWS provides and SDK for accessing its services, in our case we will use the node js SDK to upload the images to the S3 bucket and serve them to the users using the Cloudfront.

The image upload process is the process of uploading images for the products and categories in the system. The image upload process is shown in figure **??**.

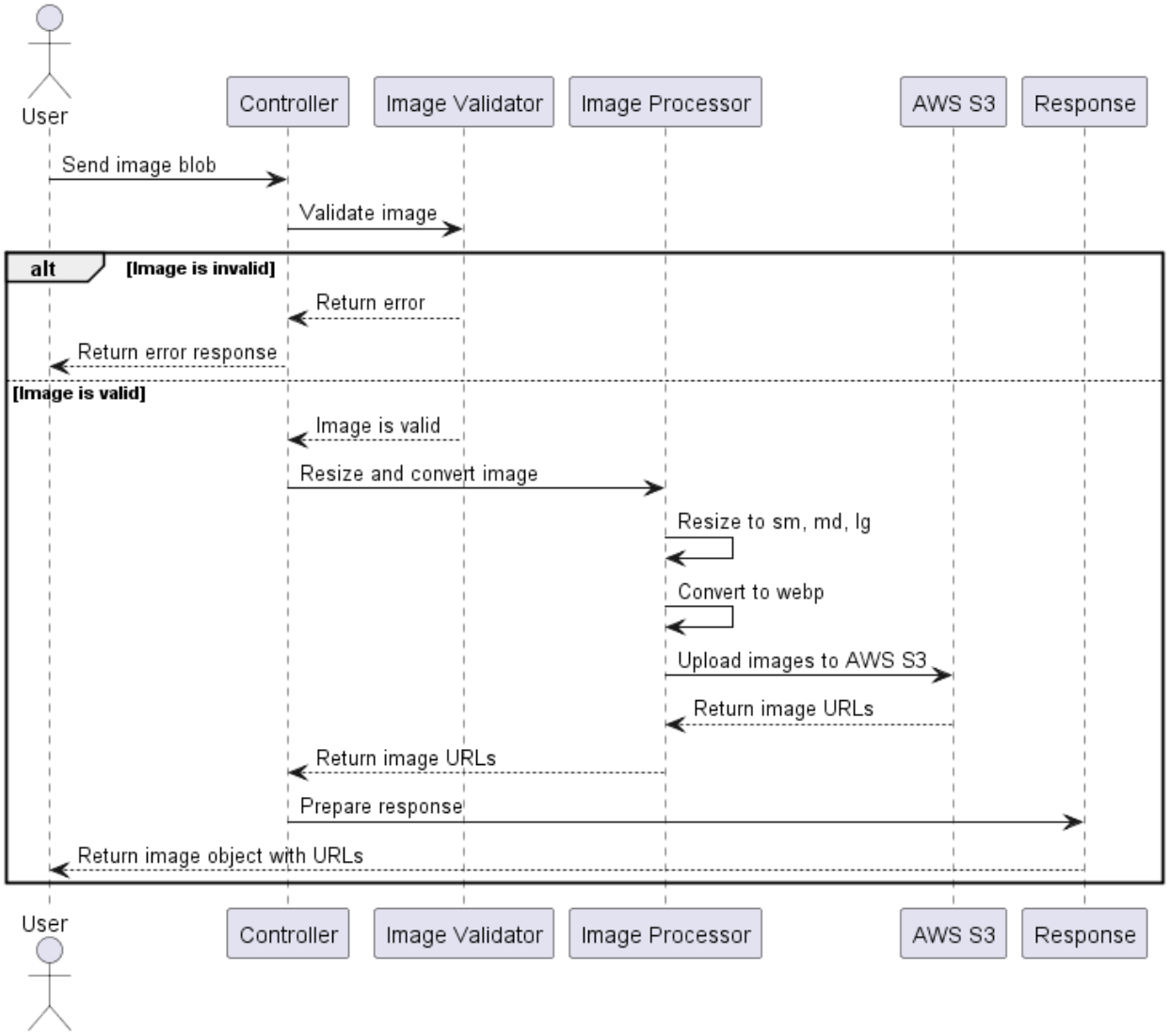



**Figure 4.5:** Image Upload Sequence Diagram

### 4.3.2 Frontend

#### 4.3.2.1 User Journey

We can summarize the user journey for the product and category management process in the system in the figure **??**.

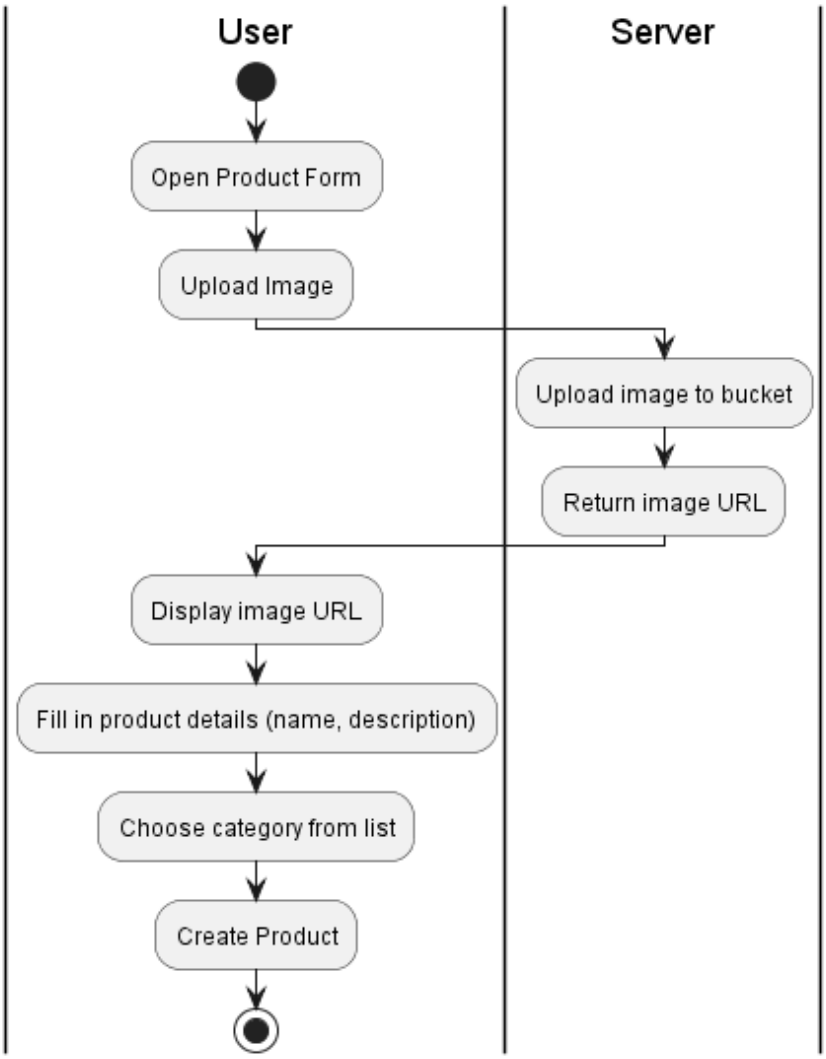



**Figure 4.6:** Product and Category User Journey

#### 4.3.2.2 UI Design

The UI design for the product table is shown in figure **??**.

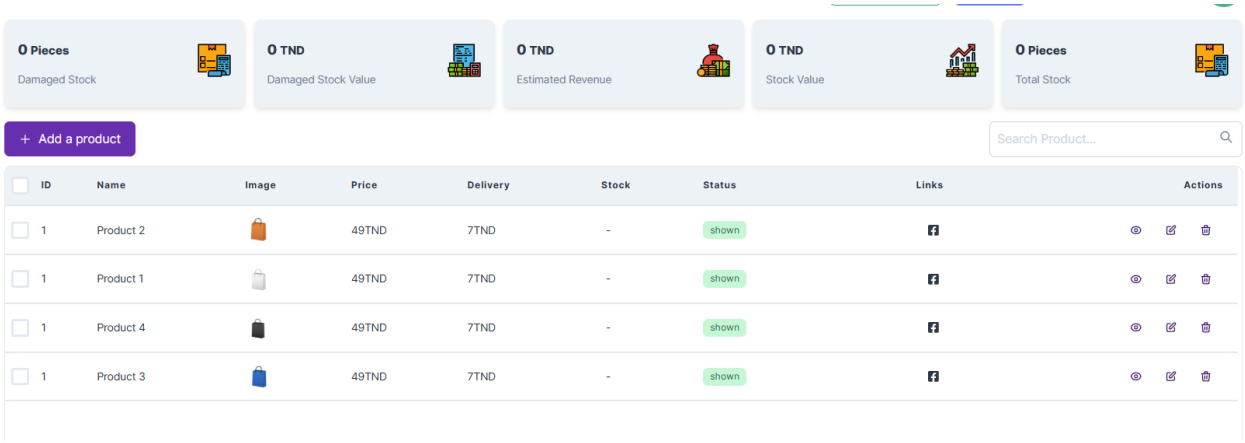



**Figure 4.7:** Product Table UI Design

The UI design for the product form is shown in figure ??.

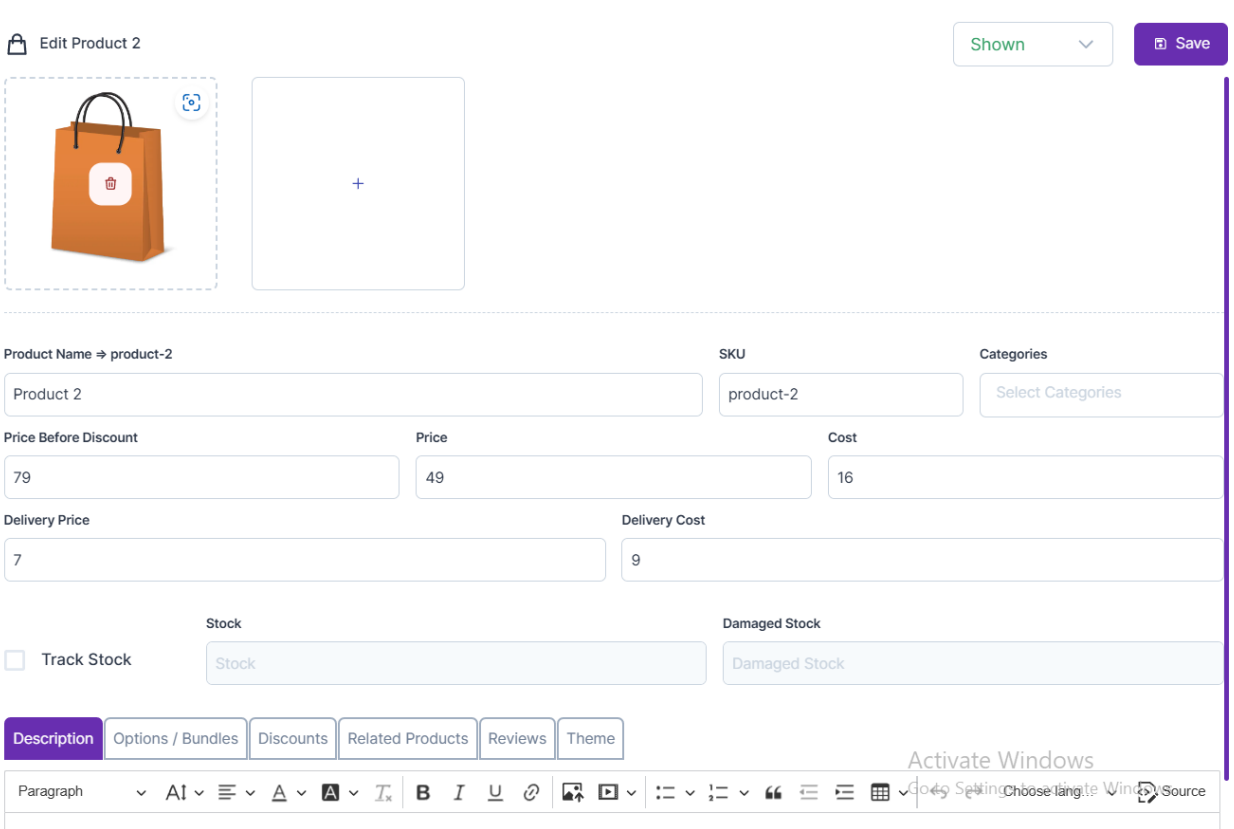



**Figure 4.8:** Product Form UI Design

We can skip the category UI design since it's similar to the product UI design.

## 4.4 Validation

### 4.4.1 Results

After implementing the product and category management process with the caching layer and the image upload process and even indexing the database we can see that the system is able to handle the CRUD operations for the products and categories and the search operations for the products and categories quickly and efficiently. The system is able to handle the image upload process and serve the images to the users in the best way possible.

### 4.4.2 Discussion

One of the main flaws in the system was invalidating the cache when creating or updating entities. The process can be easy for handling one simple entity but as the application grows bigger and the relations between the entities grow more complex the process of invalidating the cache can be more complex and error-prone. We can remidiate this issue by using a more complex and centrelised cache invalidation system that can handle the cache invalidation for all the entities in the system. this system can take only the entity name and the store ID and invalidate all the related entities in the cache. So when we add more entities or relations between the entities we don't have to worry about the cache invalidation process, we just need to update the cache invalidation system to handle the new entities and relations. And this figure describes the cache invalidation process.

## 4.5 Summary & Conclusion

In this chapter we discussed the product and category management process for the system. We discussed the design and implementation of the product and category CRUD operations, the search operations, the caching layer, and the image upload process. We also discussed the results and the issues with the system and how to fix them. In the next chapter we will discuss the order management process for the system.

*Chapter 5*

# Order and Delivery Management

## 5.1 Introduction

In this chapter we will talk about the order and delivery management process for the system. We will start by discussing the order CRUD and how we have integrated with multiple delivery services to provide the best experience for the users.

## 5.2 Sprint Backlog

**Table 5.1:** Sprint 3 backlog for Order & Delivery Management

| Sprint | User Story | ID | Task |
|---|---|---|---|
| 3 | 3.1 | 3.1.1 | Backend: Implement CRUD API endpoints for orders. |
| | | 3.1.2 | Frontend: Design order management UI for CRUD operations. |
| | 3.2 | 3.2.1 | Backend: Implement API endpoints for searching orders by multiple fields. |
| | | 3.2.2 | Frontend: Implement order search functionality by multiple fields. |
| | 3.3 | 3.3.1 | Backend: Implement functionality to link orders to delivery services. |
| | | 3.3.2 | Frontend: Develop UI for linking orders to delivery services. |

## 5.3 Design & Implementation

In this section we will talk about the design and implementation of the order and delivery management process. We will start by discussing the backend design and implementation then move on to the frontend design and implementation.

### 5.3.1 Backend

#### 5.3.1.1 Database Schema & Modelling

Before diving into the CRUD operations, let's discuss the database schema for the system. The order schema is one of the most complex schema in the system so far since everything in the platform relies on it. The tricky part here is to model the order schema is a way that allows us to fullfill the requirements of the system in all aspects. We have to make suure that the schema is flexible enough to allow us to add new features in the future. Figure **??** shows the order schema.

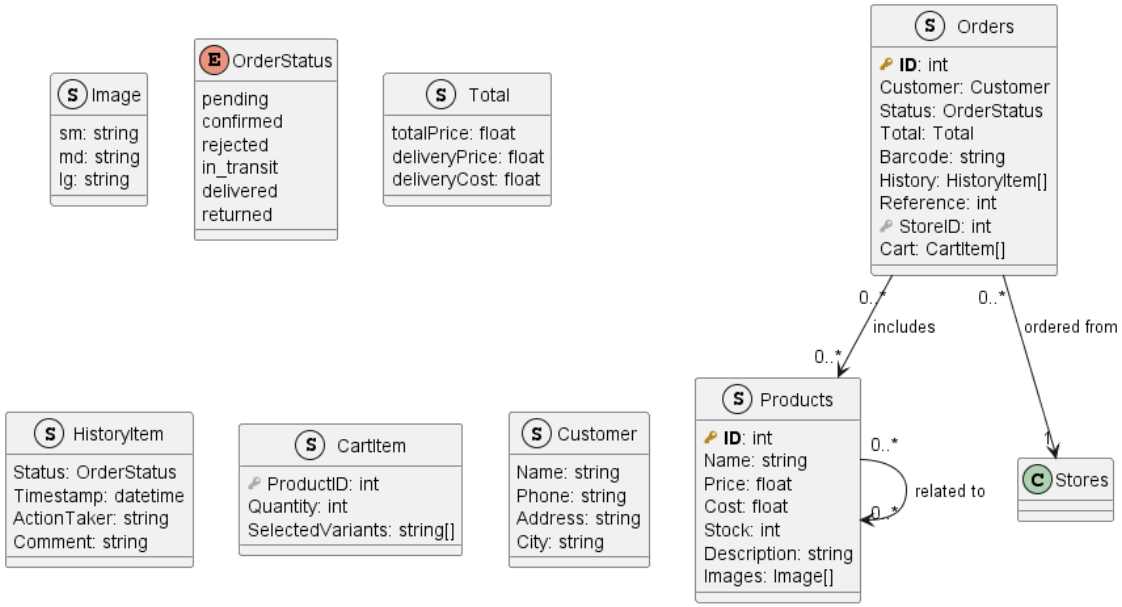



**Figure 5.1:** Order Database Schema

**Cart** To keep track of the items in the order, we have created a cart item object which was embedded inside the cart array inside the order schema. The cart item object contains the product id, quantity, and price of the product. the product id is a reference to the product schema.

**History** To keep track of the actions made on the order, we have created an array of objects that contains the status, timestamp and the user who made the action. The status is an enum that contains the possible statuses of the order.

**Customer** We kept the customer information embedded inside the order schema and we kept it unstructured for these reasons:

- The customer information is not used for any other purpose other than the order and creating a separate schema for it would be an overkill and ressource consuming. (Which we did in the past)
- Since we are working with a NoSQL database, we can keep the customer information unstructured and it will be easier to query and update and we can add new fields in the future without any issues. (Which we couldn't do in the past)
- We can index the customer field inside the order schema to speed up the search operations.

**Reference** The reference of the order is a unique identifier by store, so it should increment by the number of orders in the store. (We achieved this using mongoose pre creation hooks)

The rest of the fields are self-explanatory and we will discuss them in the CRUD operations.

#### 5.3.1.2 Search & Read Operations

**Design Logic** In this paragraph, we will discuss the design logic behind the search operations for the orders. The search operations are the operations that allow users to search for orders using multiple fields. The search operations are shown in figure **??**.

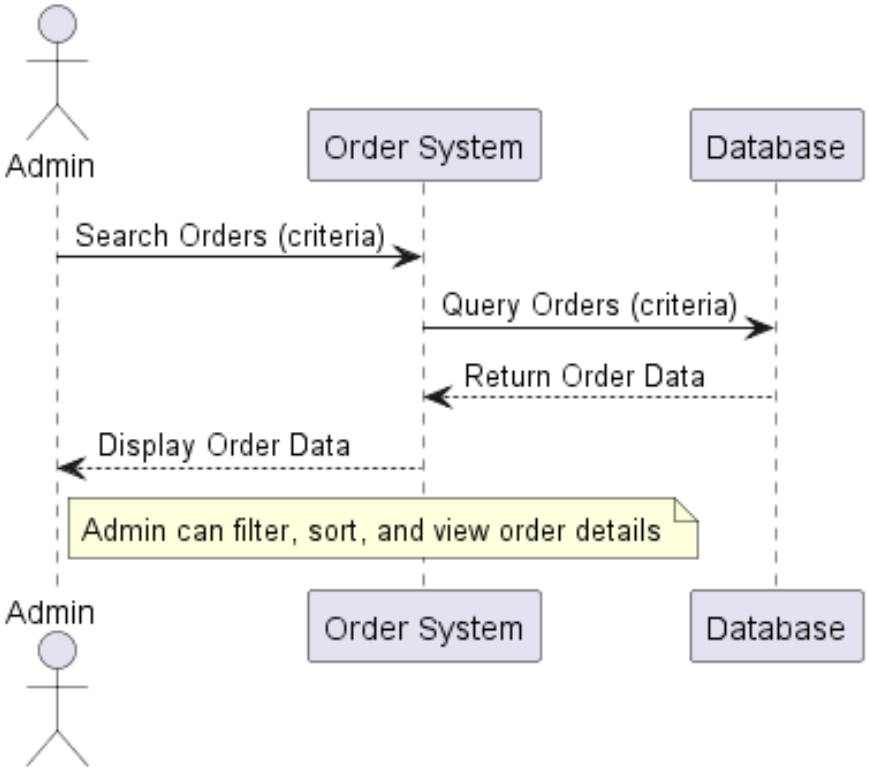



**Figure 5.2:** Order Search Sequence Diagram

**Indexing(MongoDB)** To speed up the search operations, we have created indexes on the fields that are used in the search operations. We have created indexes on the following fields: [**mongodbIndexOptimization**]

- reference
- customer
- status
- createdAt
- barcode

We should choose the fields carefully since creating indexes on all fields will slow down the write operations and consume more memory.

**Caching(Redis)** Indexing plays a crucial role in the search operations, but it will only affect the first read opertations since we implemented a caching mechanism that caches the

search results for a certain amount of time. The caching mechanism here is a bit tricky especially when choosing the key for the queries with filters. We tried to simplify the logic in this figure ??.

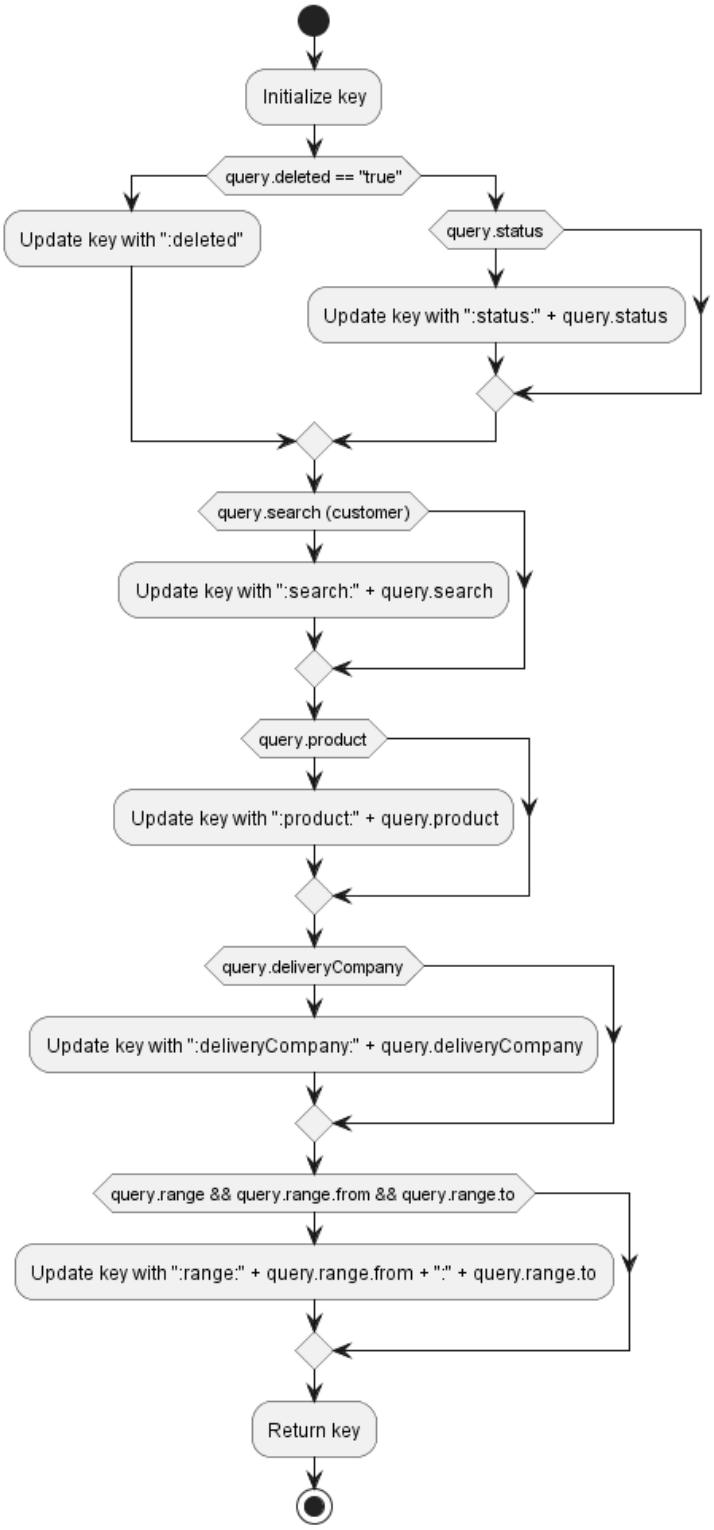



**Figure 5.3:** Order Cache Sequence Diagram

#### 5.3.1.3 Create & Update Operations

**Design Logic** Now, we will discuss the design logic behind the create and update operations which allow users to create and update orders. The create and update operations are shown in figure **??**.

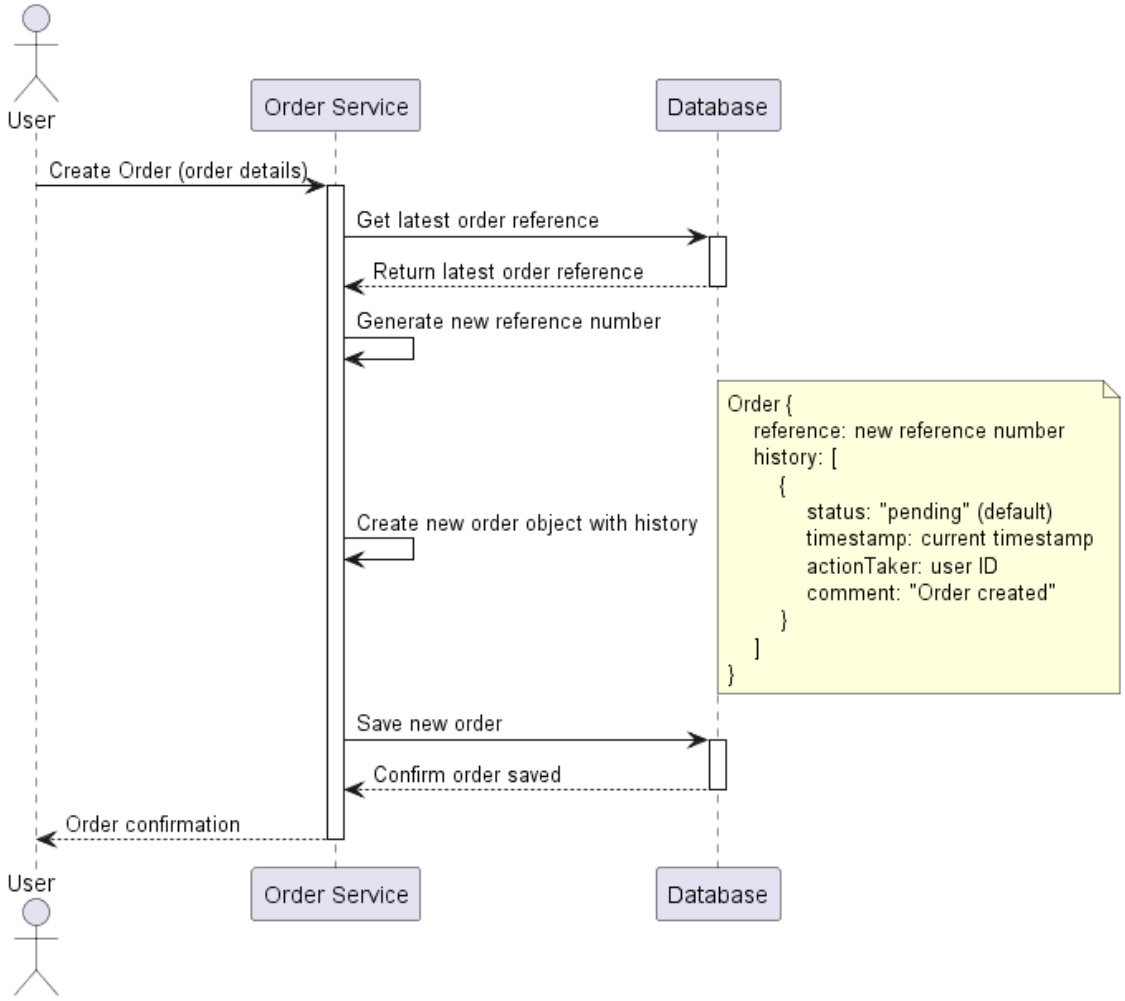



**Figure 5.4:** Order Create Sequence Diagram

The same logic goes for the update and delete operations, the history array will be updated with a new object that contains the status, timestamp and the user who made the action.

**Note** : The system never allows for order deletion, the orders are only marked with status "deleted" and they are never shown in the default search resutls unless the user specifically asks for them.

#### 5.3.1.4 Delivery Companies Integration

**Design Logic** : This is one of the most important features in the system, the delivery companies integration. The system should allow users to link the orders to the delivery services and track the delivery status of the order later. First of all, we will assume that the user has already intergrated his delivery company account with the system. The integration process is out of the scope of this chapter. To trigger the delivery process, the user should mark the order as confirmed and choose the integrated delivery company and the system will automatically send the order and save the barcode(tracking number) and mark the order as "uploaded". The delivery process is shown in figure **??**.

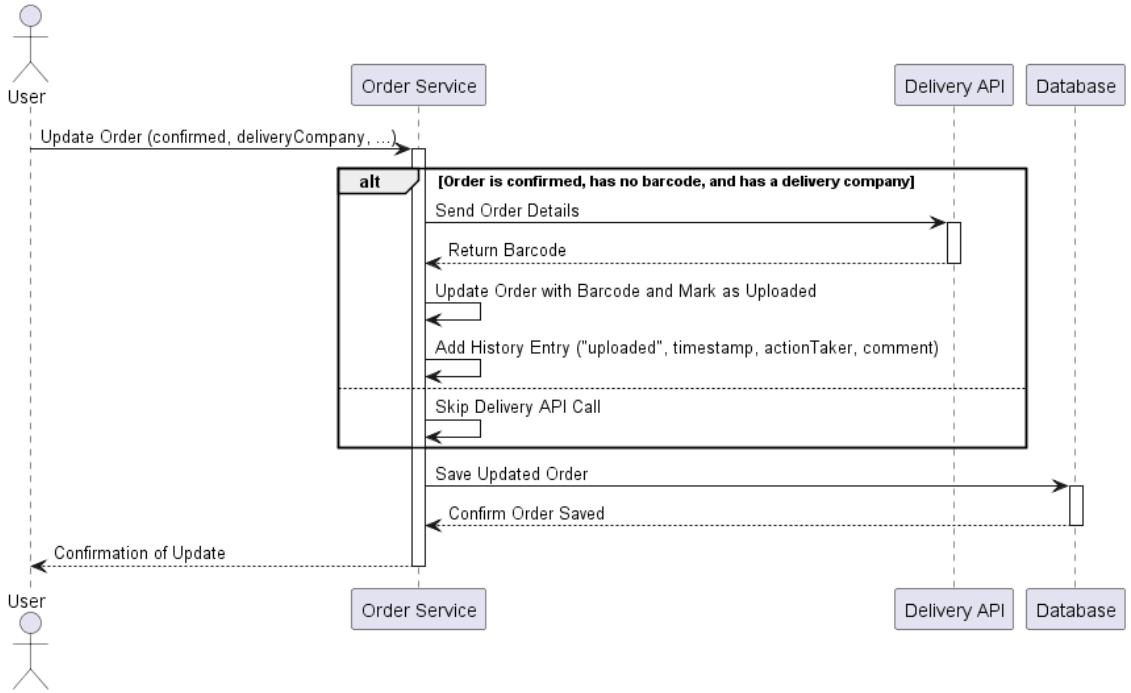



**Figure 5.5:** Order Delivery Sequence Diagram

**Implementation** : The implementation of the delivery companies integration is a bit tricky since we have to handle the errors and more than one delivery company at the same time knowing that each system has its own API and logic. We have created a delivery service using a switch statement that loads the delivery company API function based on the delivery company name and fetches the required keys from the user since these external APIs usually use an authentication logic. The delivery service is shown in figure **??**.

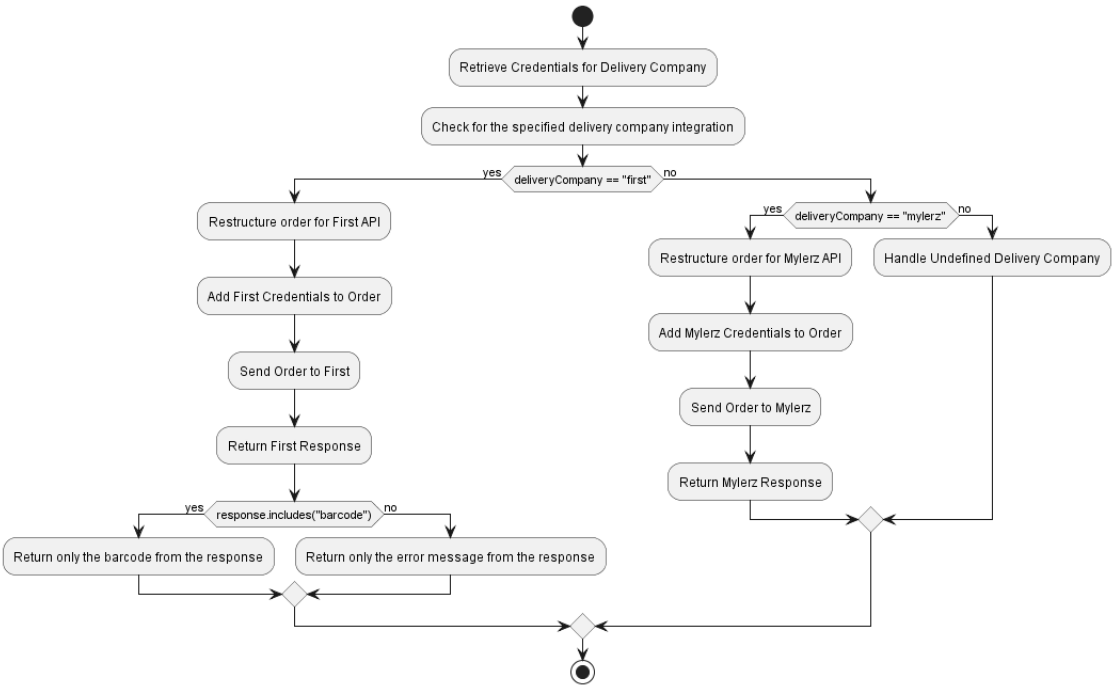



**Figure 5.6:** Delivery Service Mechanism

## 5.3.2 Frontend

### 5.3.2.1 UI Design

The UI design for the order table is shown in figure ??.

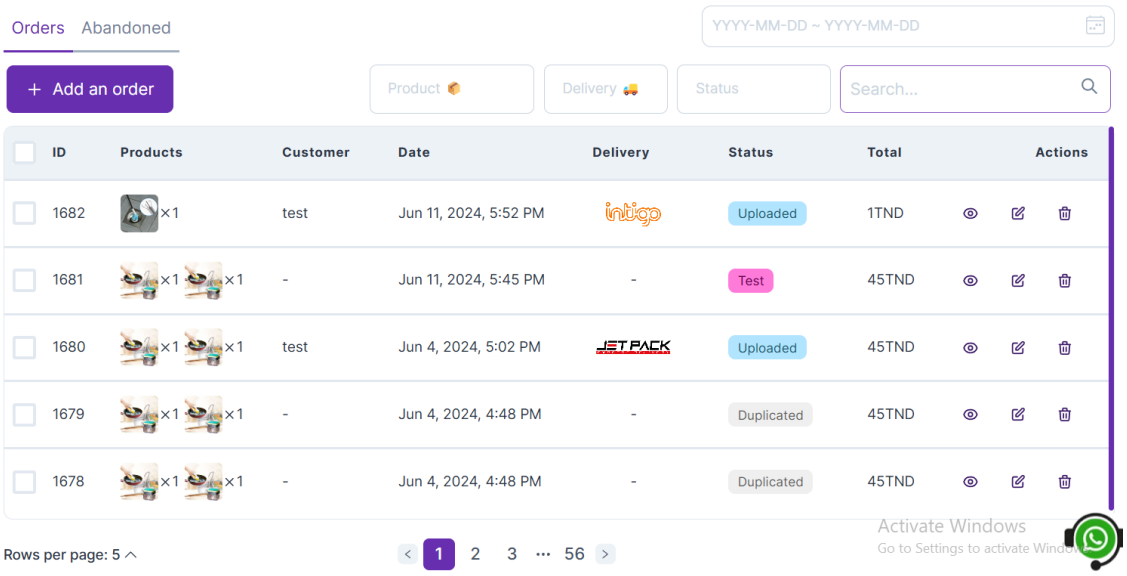



**Figure 5.7:** Order Table UI

The UI design for the order form is shown in figure ??.

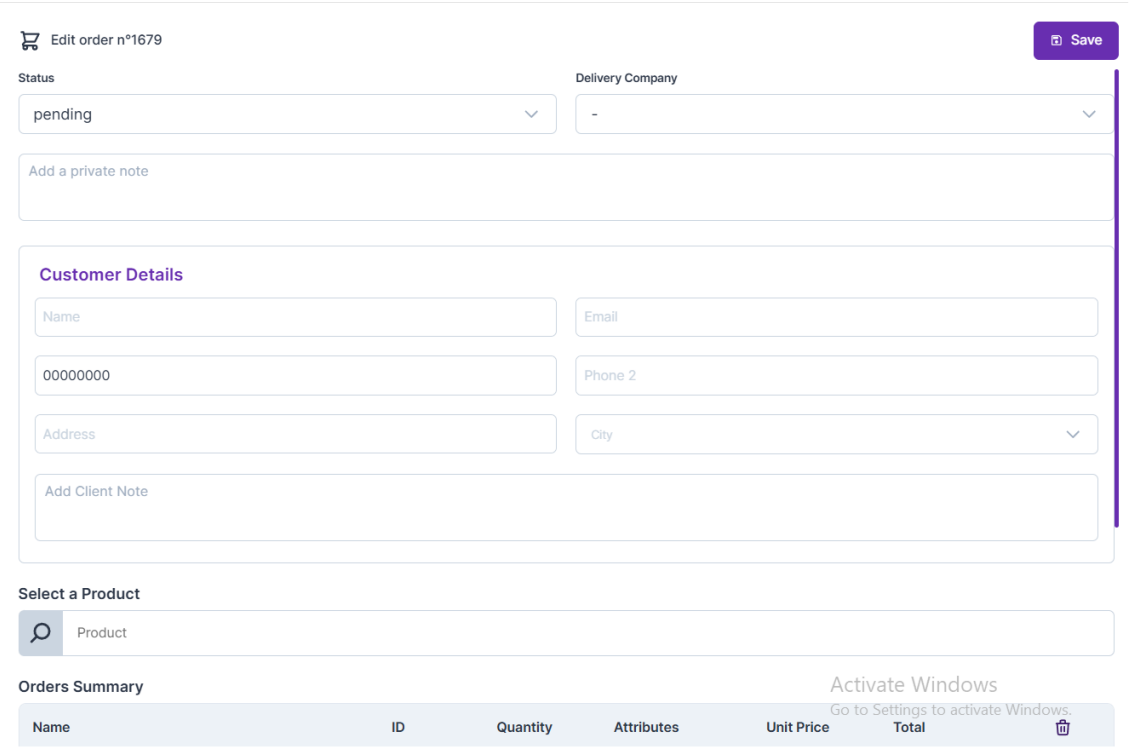



**Figure 5.8:** Order Form UI

## 5.4 Validation

### 5.4.1 Results

The results of the order and delivery management process were as expected. a sleak and fast system that allows users to create, read, update, and delete orders and link them to the delivery services. The search operations were fast and the caching mechanism worked as expected. The delivery companies integration was a bit tricky but it worked as expected.

### 5.4.2 Discussion

One of the main issues that we faced are the requirements for each delivery company external API, each company has its own logic and requirements and we had to handle them all.

## 5.5 Summary & Conclusion

In this chapter we discussed the order and delivery management process for the system. We started by discussing the requirements and then moved on to the design and implementation of the backend and frontend. We discussed the search, create, update, and delete operations and how we have integrated with multiple delivery services to provide the best experience for the users. We also discussed the validation results and the issues that we faced during the implementation.

*Chapter 6*

# Team Management & Micro-Permissions

## 6.1 Introduction

In this chapter we will talk about the team management process for the system. We will start by discussing the team CRUD and how we handle the permissions and roles for the users.

## 6.2 Sprint Backlog

**Table 6.1:** Sprint 4 backlog for Team Management

| Sprint | User Story | ID | Task |
|---|---|---|---|
| 4 | 4.1 | 4.1.1 | Backend: Implement CRUD API endpoints for managing team members. |
| | | 4.1.2 | Frontend: Design UI for managing team members (create, read, update, delete). |
| | 4.2 | 4.2.1 | Backend: Implement API endpoints for managing user permissions within the team. |
| | | 4.2.2 | Frontend: Develop UI for assigning and revoking specific permissions to team members. |

## 6.3 Design & Implementation

### 6.3.1 Backend

#### 6.3.1.1 Database Schema

For the team/staff schema we have created a new schema that contains the user id,username, password, the store id, the role, and the permissions. The idea behind creating a new schema is simply concern separation. We want to keep the user schema as clean as possible and we want to keep the team/staff schema as clean as possible. The team/staff schema is shown in figure **??**.

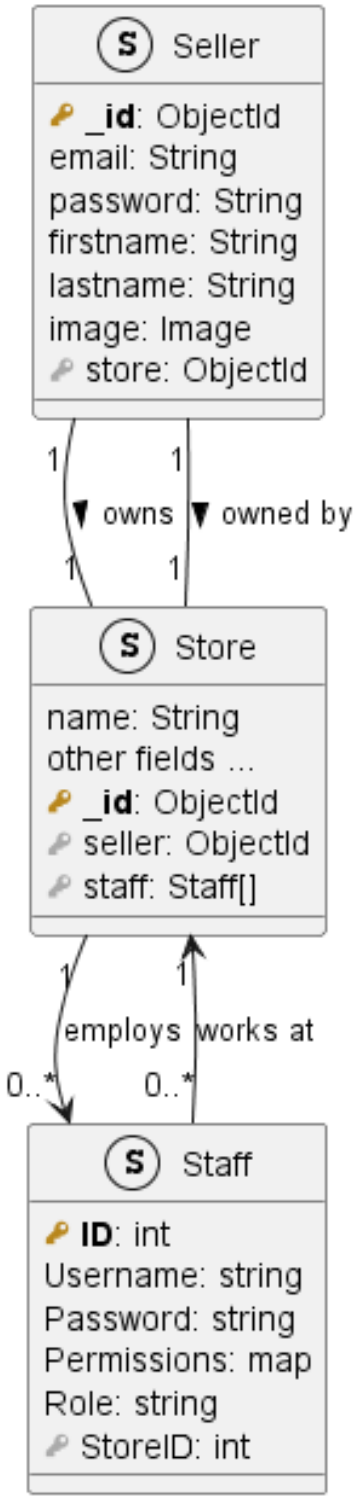



**Figure 6.1:** Team Database Schema

#### 6.3.1.2 Authentication - JWT

**Design** For the authentication part, we had to alter our login mechanism a bit so we can satisfy both ends. The staff members should be able to login using a username which is unique

to the staff schema and non-email. This choice was made to make the whole authentication process easier (Signup / Staff Creation / Login) as the staff members are not really required to have an email. In this figure we will show the login process work.

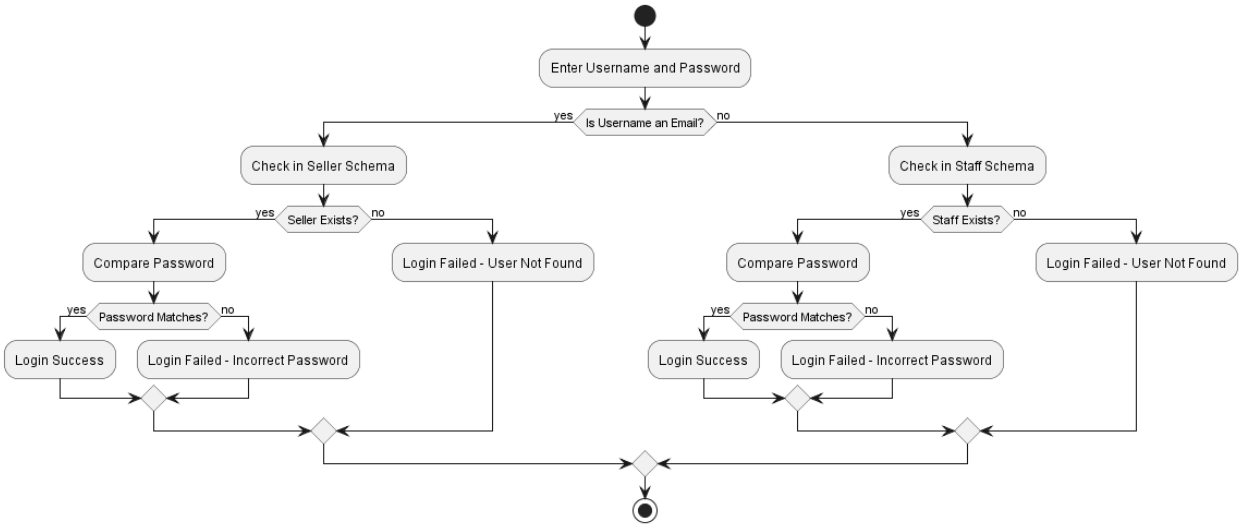



**Figure 6.2:** Login Sequence Diagram

**Implementation** In the implementation part we will use a code snippet to show how we

```
router.post('/login', async (req, res) => {
    const { username, password } = req.body;
    if (!username || !password) {
        return res.status(400)
            .json({ message: 'Username and password are required' });
    }
    // if the username is email
    if (username.includes('@')) {
        // seller logic here
    } else {
        // staff logic here
    }
});
```

Listing 6.1: Login Process Implementation

#### 6.3.1.3 Authorization - JWT

**Design** For the authorization part, we had to alter our middleware logic so it can accept the staff schema as well as his permissions. The idea is to decode the token and check if the user is a staff member or a seller. If the user is a staff member we will check the permissions that the controller needs to be accessed. **??** shows the authorization process work.

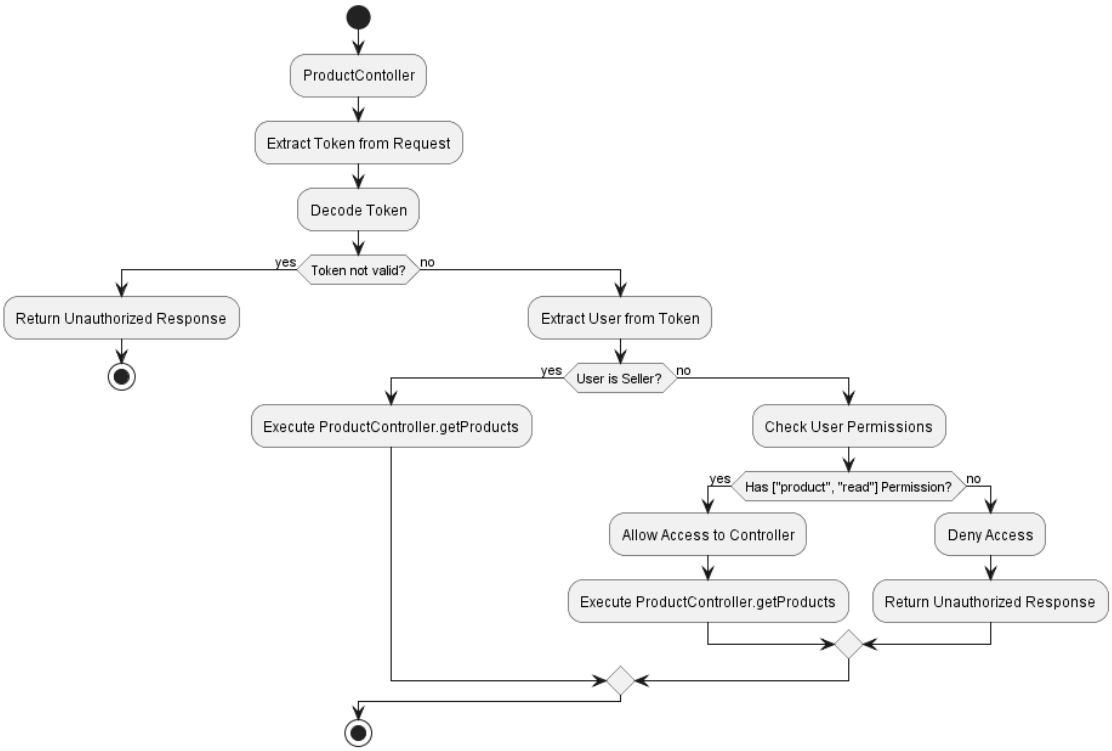



**Figure 6.3:** Authorization Sequence Diagram

**Implementation** In the implementation part we will use a code snippet to show how we implemented the authorization process.

```
1        router.get(
2            "/",
3            auth({
4                Controller: ProductController.getProducts,
5                Permission: ["product", "read"],
6            })
7        );
```

Listing 6.2: Login Process Implementation

### 6.3.2 Frontend

#### 6.3.2.1 UI Design

The UI design for the team table is shown in figure ??.

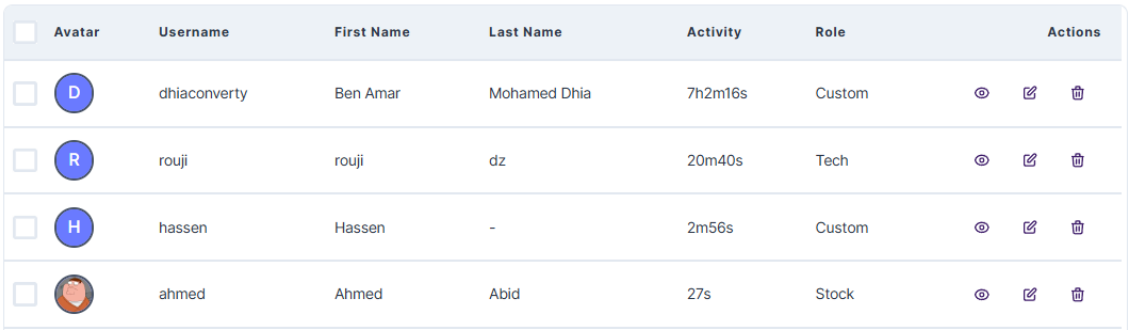



**Figure 6.4:** Team Table UI

The UI design for the team form is shown in figure ??.

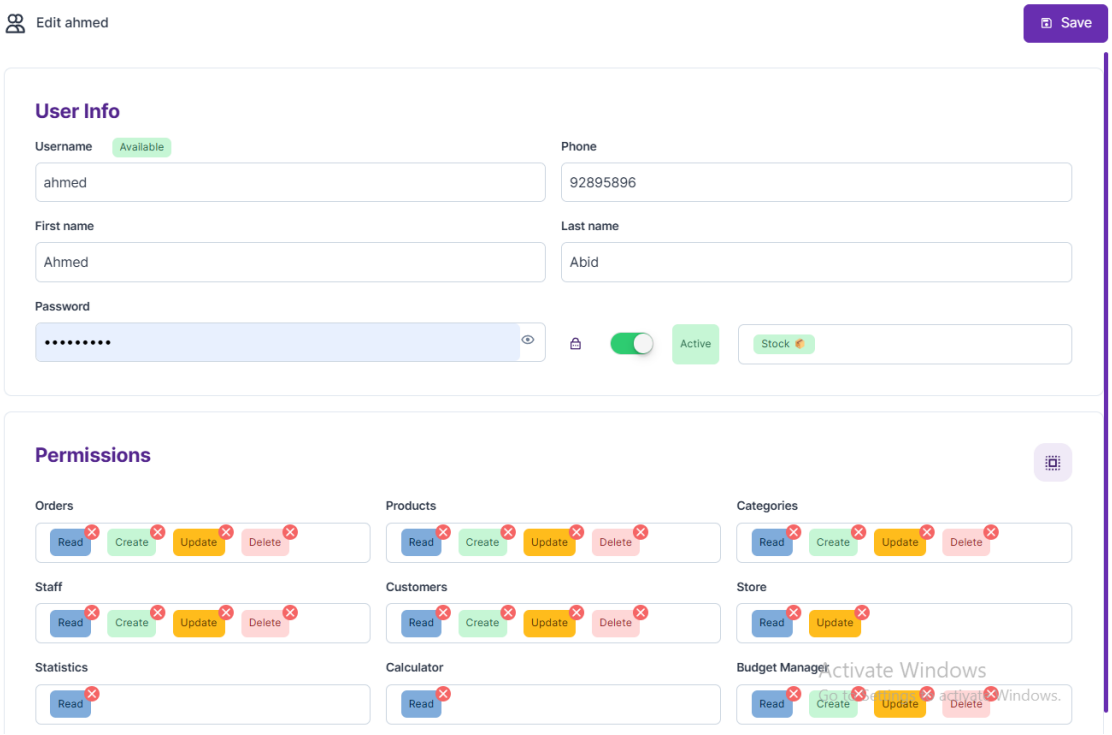



**Figure 6.5:** Team Form UI

## 6.4 Validation

### 6.4.1 Results

The team management process was a success. The users were able to create a team and add users to the team. The users were also able to give specific permissions to the users in the team. The users were also able to revoke the user's permissions quickly.

### 6.4.2 Discussion

We could have improved the team management process by adding a feature that allows the users to log the actions made for the last 24hours.

## 6.5 Summary & Conclusion

In this chapter we talked about the team management process for the system. We started by discussing the team CRUD and how we handle micro permissions for the users.

*Chapter 7*

# Realtime Synchronization Using WebSockets

## 7.1 Introduction

In this chapter we will talk about the realtime feature for the system. We will start by discussing how we implemented the realtime feature and how we handle the permissions for the users.

## 7.2 Sprint Backlog

**Table 7.1:** Sprint 5 backlog for Realtime Feature

| Sprint | User Story | ID | Task |
|---|---|---|---|
| 5 | 5.1 | 5.1.1 | Backend: Implement websocket API for realtime updates. |
| | | 5.1.2 | Frontend: Integrate websocket client to receive and display realtime updates. |
| | 5.2 | 5.2.1 | Backend: Implement permission-based socket connections. |
| | | 5.2.2 | Backend: Implement functionality to quickly revoke user permissions. |

## 7.3 Design & Implementation

In this section we will talk about the design of the system in general. We will start by discussing the backend design and how we implemented the realtime feature.

### 7.3.1 Backend

#### 7.3.1.1 Socket io

For the realtime feature we have used the socket io library. The idea behind using the socket io library is to make the system more responsive and to make the system more interactive. The socket io library is a library that allows us to create a websocket connection between the client and the server. [**socketioChatApp**]

#### 7.3.1.2 Rooms & Permissions

**Design** As we all know, the socket io library allows us to create rooms. The idea behind creating rooms is to group the users based on their permissions. For example we can create a room for products, a room for orders and a room for categories. The idea behind creating rooms is to make the system more scalable and to make the system more secure.

**Implementation** Below is a code snippet that details the implementation of the rooms and permissions. (Staff logic)

```
import { Server, Socket } from "socket.io";

const io = new Server();

io.on("connection", (socket: Socket) => {
    socket.on("join", (room: string) => {
        let token = tokenFromHeaders(socket.handshake.headers);
        if (token) {
            let permissions = getPermissions(token);
            if (permissions.order.read) {
                socket.join(storeId + ":order")
            }

            if (permissions.product.read) {
                socket.join(storeId + ":product")
            }

            //etc ...
        }

    });
});
```

Listing 7.1: Websocket: Rooms & Permissions Implementation

#### 7.3.1.3 Emitting Events

**Design** After setting up the rooms, we can now implement the realtime logic for the system. The idea behind emitting events is to notify the users in the same room about the changes made by other users. The emitting events are shown in figure **??**.

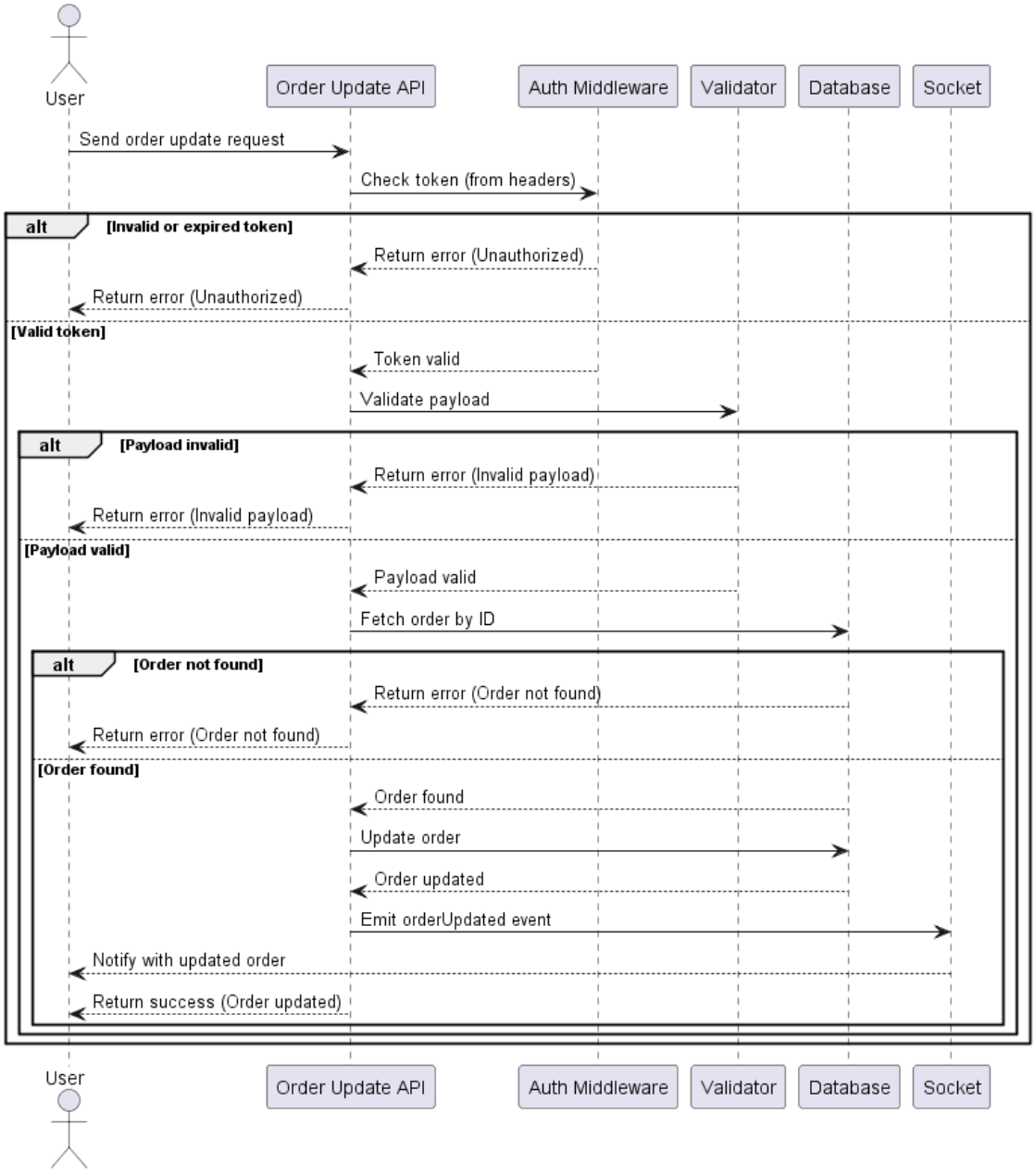



**Figure 7.1:** Emitting Events

**Implementation** In the implementation part we will use a code snippet to show how we implemented the emitting events. (Order Controller)

```
import { Server, Socket } from "socket.io";

const io = new Server();

//pass the instance to the order controllers
app.use("/api/v1/order", OrderRouter(io));

...
const updateOrder = (io: any) => async (req: Request, res: Response) =>
...
// at the end of the function
io.to(storeId + ":order").emit("order:updated", order);
```

Listing 7.2: Websocket: Emitting Events Implementation

#### 7.3.1.4 Scaling the System

**Design** After implementing the realtime feature, we have to make sure that the system can handle the load. The issue here is that our backend is stateless whereas the socket io library is stateful. So we can not really scale the system horizontally. Hopefully, the socket io library has a solution for this issue. The solution is to use the redis adapter. The redis adapter is a library that allows us to scale the system horizontally. The redis adapter is shown in figure ??.

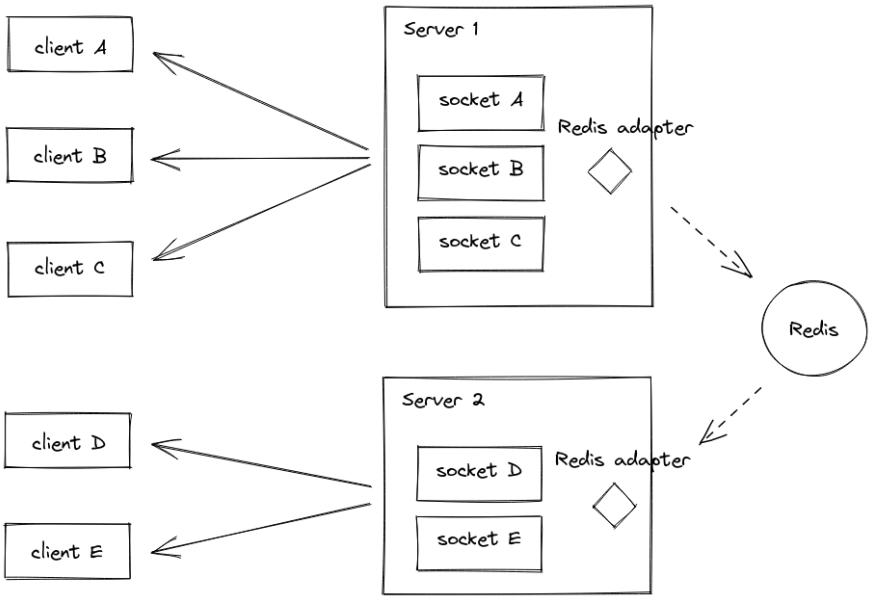



**Figure 7.2:** Redis Adapter

**Implementation** In the implementation part, we just need to add the redis adapter connection to our socket io setup.

```
import { redis } from "./Database";
import { Server, Socket } from "socket.io";
import { createAdapter } from "@socket.io/redis-adapter";

const io = new Server();

io.adapter(createAdapter(redis, redis.duplicate()));

...
```

Listing 7.3: Websocket: Redis Adapter Implementation

### 7.3.2 Frontend

The socket io connection in the frontend is pretty straightforward. We just need to connect to the socket server and handle the events.

## 7.4 Validation

### 7.4.1 Results

The implementation of the realtime feature was successful. The system can now handle the realtime feature and failover gracefully. The system can also allow socket connections based on the user's permissions. The system can also revoke the user's permissions quickly.

### 7.4.2 Discussion

The only thing that we missed here is the reconnection logic. The Socket.io library has a plugin that handles the reconnection logic but it is not very reliable. We have to implement our own reconnection logic to make sure that the system can handle the reconnections gracefully. For now we stick with reloading the page everytime the connection is lost.

## 7.5 Summary & Conclusion

In this chapter, We have talked about the realtime feature for the system. We have discussed how we implemented the realtime feature and how we handle the permissions for the users. We have also discussed the frontend part and how we implemented the realtime feature for the users. In the next chapter, we will discuss how we connected the socket server with the other components/services of the system.

*Chapter 8*

# Order Tracking Service

## 8.1 Introduction

In this chapter, we will discuss how we implemented the order tracking system using another service running periodically to update the order status.

## 8.2 Sprint Backlog

**Table 8.1:** Sprint 6 backlog for Order Tracking System

| Sprint | User Story | ID | Task |
|---|---|---|---|
| 6 | 6.1 | 6.1.1 | Backend: Implement a service to periodically update order statuses. |
| | | 6.1.2 | Backend: Ensure real-time updates for order status changes. |
| | 6.2 | 6.2.1 | Frontend: Display real-time order status updates to users. |
| | | 6.2.2 | Backend: Ensure the order tracking service handles failover gracefully. |

## 8.3 Design & Implementation

In this section, we will talk about the design of the system in general. We will start by discussing the backend design and how we implemented the order tracking feature.

### 8.3.1 Backend

#### 8.3.1.1 Setup

We choose to seperate our order tracking module in another service for these reasons:

- To make the system more performant.
- To get rid of single points of failure.
- To make the system more scalable.

We chose to use go for the order tracking service since it is a high performance language and it is easy to write concurrent code in go. [**goCoolMicroservices**]

#### 8.3.1.2 Order Tracking Logic

**Design** Since tunisian delivery companies do not use a webhook system to update the order status, we have to implement a polling system to update the order status. The idea behind the polling system is to check the order status every 1 hour. The polling system is shown in figure **??**.

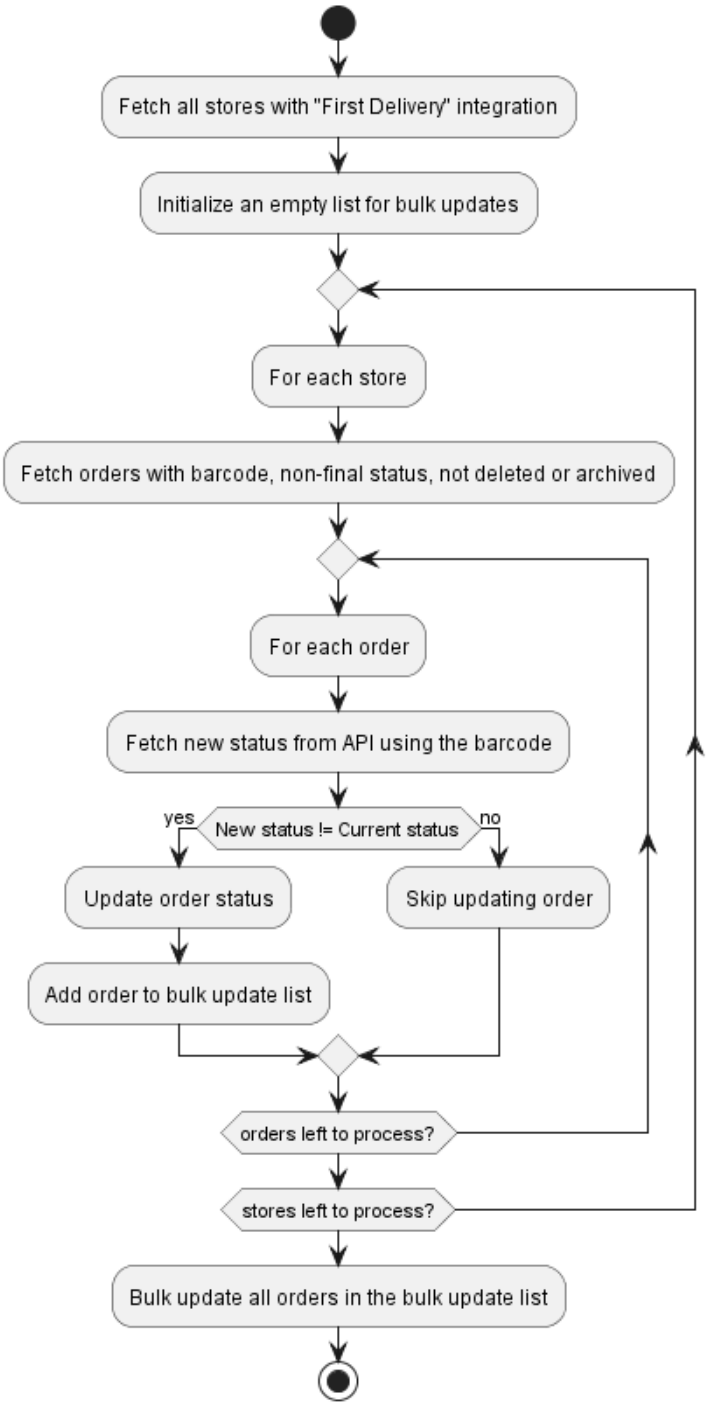



**Figure 8.1:** Polling System Flow Diagram

To improve performance, we used bulk updates feature in mongo drivers, where you can update multiple documents at once using a single query.

#### 8.3.1.3 Post-update Logic

After updating the orders, we have to take in count two things:

- Invalidate the cache for the orders.
- Notify the users about the order status change via the socket.

**Caching Invalidation** Since we are using go for the order tracking service, we can use the redis library to invalidate the cache for the orders. we just need to know the key pattern for the orders. We can follow these steps:

- Get the store id from the order.
- List all the keys that have prefix `store:store_id:orders`.
- Delete all the keys.

**Socket Notification** The socket notification is a bit tricky here. We have to options:

- Connect to the socket server from the go service directly.
- Use an internal connection mechanism to notify the main backend about the order status change.

Since there is no Socket io library for go, we have to use the second option. We can use the rabbitmq to notify the main backend about the order status change. RabbitMQ is a message broker [**rabbitmqForBeginners**] that allows us to send messages between services. The logic is shown in figure, we keep the producer on the order tracking service and the consumer on the main backend. The logic is shown in figure **??**.

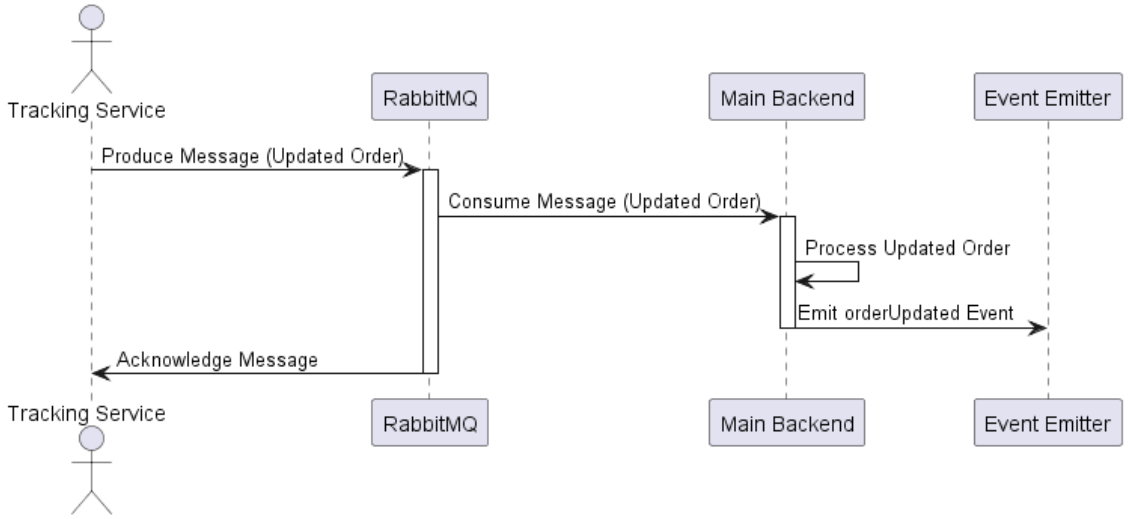



**Figure 8.2:** RabbitMQ Logic

### 8.3.2 Frontend

#### 8.3.2.1 User Journey

There is no user journey for this feature since it is a background service.

#### 8.3.2.2 UI Design

Same thing the UI design. We can only provide this figure fig:order-history to show the history of an order.

| Date | Comment | Action by | Status |
|---|---|---|---|
| Mar 28, 2024, 7:25 PM | - | Customer | Pending |
| Mar 29, 2024, 10:23 AM | - | bruce | Confirmed |
| Mar 29, 2024, 10:23 AM | - | bruce | Uploaded |
| Apr 4, 2024, 3:23 PM | - | sami@converty.shop | Packed |
| Apr 4, 2024, 5:15 PM | - | mylerz | In transit |
| Apr 5, 2024, 1:17 PM | - | mylerz | Delivered |
| Jun 17, 2024, 9:55 PM | auto archived | Converty | Archived |

Activate Windows
Go to Settings to activate Windows.

**Figure 8.3:** Order History

## 8.4 Validation

### 8.4.1 Results

This feature was tested by the first users of the platform. The results were satisfying. The order status was updated in real-time and the users were happy about it.

### 8.4.2 Discussion

One of the main issues with this feature is the polling system. The polling system is not really efficient. We have to check the order status every 1 hour. This is not really efficient. We have to find a better solution for this issue like using a webhook system. Another issue is the Delivery systems that use a single tracking order API. This created an issue on our end since we can't really make an api call for each order as it will affect the performance of both systems. (Average 30 orders for each store)

## 8.5 Summary & Conclusion

In this chapter, we have discussed the order tracking feature for the system. We have discussed how we implemented the order tracking feature and how we handle the order status change. We have also discussed the results and the issues with the system and how to fix them.

*Chapter 9*

# Conclusion

## 9.1 General Conclusion

In this internship, we had the opportunity to work on a real-world project. We learned a lot about the software development process and how to work in a team. We also learned how to use different technologies and tools to build a scalable and performant system. We faced many challenges during the internship, but we managed to overcome them by working together and learning from our mistakes. We are proud of what we have accomplished during this internship, and we are excited to see the project go live.

Firstly, we started by understanding the requirements of the project and the technologies we would be using. We then designed the architecture of the system and implemented the core features of the user onboarding process.

Secondly, we talked about the product and the categories schemas and how we used caching to improve the performance of the system. We also discussed the implementation of the search functionality.

Then, we discussed how we can implement delivery companies APIs to send orders and later to track them.

Finally, we implemented real-time notifications using WebSockets and discussed how we can improve the user experience in the platform by adding more features such as order dispatching and two-factor authentication.

## 9.2 Perspectives and Future Work

There are many ways in which the project can be improved in the future. Here are some of the ideas that we have for future work:

- **Improve the user onboarding process:** We can add more features to the user onboarding process, such as social login, email verification, and password recovery.
- **Improve the search functionality:** We can improve the search functionality by adding more filters and sorting options.

- **Improve the order module:** We can add more features in orders module such as order dispatching for users.
- **Improve the security of the system:** We can improve the security of the system by adding more security features, such as two-factor authentication, rate limiting, and IP whitelisting.